Problem Chosen	IM^{2}C 2026 International Round Summary Sheet	Team Control Number
Only		26672879

Protecting Wildlife at Scale: A Modeling Approach for Etosha National Park

Summary

Large reserves like Etosha face limited personnel and technology. The IMMC requested a model to define protection, allocate resources, and assess protection and staffing over time. We address this using Etosha as a case study, with methods adaptable to other reserves.

For Task 1, we discretize the park into a 4 km grid and assign each cell four risk scores (human threat, natural threat, species vulnerability, coverage gap). We define combined risk and protection per cell; a cell is protected if risk $\leq \tau$. With $\tau = 0.6$ we get 1,272 cells (720 protected, 552 not) and maps of dominant risk type (H/N/S/C) and protection status.

For Task 2, we allocate patrol, fence, and monitoring to cells under budgets using a risk–resource weight matrix and a greedy rule (cells by priority, each gets the best remaining resource for its dominant risk). With illustrative budgets we assign resources to about 260 cells and produce an allocation map and CSV.

For Task 3, we use a protection–threat–effort ODE ($P(t)$, $T(t)$, effort E; $E = 1 \approx 295$ staff). We simulate trajectories and derive steady-state effort $E^* = (\gamma/\delta)(1-\bar{P})$. Results: time-series plots, E^* vs. $\bar{P}$, and indicative staffing (e.g., 60% protection $\Rightarrow$ $E^* \approx 0.16$, $\sim$47 staff).

For Task 4, we scale ODE parameters by 0.8–1.2 and report E^* for $\bar{P} = 0.6$, and run scenarios with $E = 1, 0.7, 0.5$ to compare $P(t)$. We obtain sensitivity and scenario figures that quantify robustness and the effect of resource cuts.

For Task 5, we summarize strengths (quantitative protection, four risk dimensions, strategic allocation, ODE, sensitivity/scenarios, OSM data) and limitations (aggregate dynamics, scalar effort, deterministic, uncalibrated, OSM-dependent, no cost constraints), and discuss applicability.

For Task 6, we adapt the model to Yellowstone and the Sundarbans: same structure, updated inputs (boundaries, roads/waterways, gates, threats, species), required data, and expected shifts in metrics and deployment, giving a reusable adaptation plan.

For Task 7, we write a two-page non-technical letter to the IMMC summarizing the approach, recommendations, and insights for decision-makers.

In summary, we deliver a full pipeline from risk definition through allocation, dynamics, sensitivity, limitations, adaptation, and a non-technical letter, for both technical review and conservation decision-making.

Keywords: Wildlife Conservation; Etosha National Park; Spatial Risk Prioritization; Strategic Resource Allocation; Protection–threat Dynamics

Contents

1 Introduction **4**
- 1.1 Problem Background . . . 4
- 1.2 Task Restatement . . . 4
- 1.3 Task Analysis . . . 5

2 Assumptions and Explanations **6**

3 Notations **7**

4 Data Acquisition and Processing **8**
- 4.1 Data Source and Acquisition . . . 8
- 4.2 Indicator Selection . . . 8
- 4.3 Processing Methods . . . 8
- 4.4 Base Map and Outputs . . . 9

5 Task 1: Priorities and Defining Protection **9**
- 5.1 Spatial Discretization and Data Sources . . . 9
- 5.2 Four Risk Dimensions . . . 9
- 5.3 Definition of Protection . . . 10
- 5.4 Results and Maps . . . 11

6 Task 2: Resource Allocation Model **12**
- 6.1 Grid and Risk Inputs from Task 1 . . . 12
- 6.2 Resource Types and Risk–Resource Weights . . . 12
- 6.3 Allocation Model . . . 12
- 6.4 Implementation and Results . . . 13

7 Task 3: Protection over Time and Staffing Need **14**
- 7.1 Link to Tasks 1 and 2 . . . 14
- 7.2 Protection–Threat–Effort ODE . . . 14
- 7.3 Steady-State Staffing Need . . . 14
- 7.4 Numerical Results and Figures . . . 15

8 Task 4: Sensitivity and Scenario Analysis **16**
- 8.1 Sensitivity Analysis: Parameters and Required Effort . . . 16
- 8.2 Scenario Analysis: Reduced Personnel and Effective Effort . . . 17

9 Task 5: Model Strengths and Limitations **17**
- 9.1 Strengths . . . 17
- 9.2 Limitations and Restrictive Assumptions . . . 18
- 9.3 Applicability . . . 19

10 Task 6: Adaptation to Other Protected Areas **19**
- 10.1 Choice of Sites . . . 19
- 10.2 Components That Remain Unchanged . . . 19
- 10.3 Inputs and Parameters To Modify . . . 20
- 10.4 Data Required for Calibration . . . 20
- 10.5 Expected Shift in Protection Metrics and Resource Deployment . . . 21

Letter to the IMMC **22**

References **24**

A Code **24**
A.1 Data Processing 24
A.2 Task 1: Grid and Risk 29
A.3 Task 2: Resource Allocation 33
A.4 Task 3: Protection Dynamics 36
A.5 Task 4: Sensitivity 38

B Use of AI **39**

1 Introduction

1.1 Problem Background

National parks and large wildlife reserves play a critical role in protecting endangered plant and animal species. Their size and terrain, however, often make continuous monitoring difficult. Etosha National Park in Namibia is a prominent example: it spans approximately 22,935 km^2 and includes open savanna, salt pans (such as the roughly 4,800 km^2 Etosha Pan), numerous waterholes, extensive road networks, and multiple access points [3]. Despite the presence of trained rangers, limited personnel and resources make it challenging to prevent illegal activities such as poaching and unauthorized entry [8, 9]. Technologies—including aerial drones, satellites, fixed monitoring systems, and AI-assisted detection—offer opportunities to support ranger efforts, but they also require coordination, staffing, and thoughtful deployment to be effective. Conservation organizations must therefore decide how to balance human patrols and technology to provide meaningful protection across space and time.

The International Mission for Monitoring Conservation (IMMC) has asked our team to develop a modeling approach to help guide protection strategies for large wildlife reserves and national parks, beginning with Etosha National Park as a case study. Our work must consider how limited human and technological resources can be coordinated to protect wildlife across the park over time, and how "protection" can be defined and measured in a realistic, practical, and quantitative way.

1.2 Task Restatement

The IMMC requirements are restated as follows:

1. **Conservation priorities and defining protection.** Identify and prioritize the major conservation challenges faced by the park. Risks may differ by species, habitat, location, or type of threat (human-driven and natural, e.g., poaching, wildfires, human–wildlife interaction). Using these priorities, clearly define what it means, in practical terms, for wildlife in the park to be "protected." The definition may vary across species or regions and must be quantitatively measurable within our modeling approach.

2. **Resource allocation model.** Develop a mathematical model to allocate protection resources (personnel, patrols, monitoring technologies, or other interventions) to address the identified conservation priorities. The model should account for limited resources, geographic considerations, and uncertainty, and should emphasize strategic deployment rather than complete coverage.

3. **Protection over time and staffing need.** Use the modeling approach to analyze how well the park is protected over time and to estimate the number of human personnel required to maintain the chosen level of protection. Resource use, feasibility, and practicality should be considered.

4. **Sensitivity and scenario analysis.** Investigate how changes in assumptions or available resources (e.g., reduced personnel, fewer technological assets, or altered patrol schedules) affect the protection strategy.

5. **Model strengths and limitations.** Discuss the strengths and limitations of the approach, including assumptions that may restrict its applicability.

6. **Adaptation to other protected areas.** Demonstrate how the modeling approach can be adapted for use at two other national parks or wildlife reserves on different continents. Provide an adaptation plan addressing: which components remain unchanged; which inputs or parameters need to be modified (terrain, access points, road networks, threat types, species priorities, detection capabilities); what data would be required to calibrate modified inputs; and how protection metrics and resource deployments would be expected to shift.

7. **Letter to the IMMC.** Write a two-page, non-technical letter communicating the approach, recommendations, and key insights to decision-makers.

1.3 Task Analysis

1. **Requirement 1.** We will structure conservation challenges along four dimensions: *species vulnerability* (abundance and recovery capacity), *habitat type* (self-regulation and exposure to threats), *human-induced threat* (e.g., proximity to roads, gates, and access points), and *natural threats* (e.g., wildfires, drought). We will discretize the park into spatial units (e.g., grids), assign each unit a dominant risk type, and define "protection" in terms of a quantitative risk or protection index (e.g., risk below a threshold or protection level above a threshold), which may vary by species or region.

2. **Requirement 2.** We will build a resource-allocation model that matches resource types (e.g., patrols, physical barriers, monitoring technology) to the dominant risk in each spatial unit. Different resources will be assigned different effectiveness weights for each risk type (e.g., physical barriers weighted highly for human intrusion). The model will maximize a protection objective subject to resource and geographic constraints.

3. **Requirement 3.** We will use the allocation results and a simple dynamic or time-averaged view of coverage to assess protection over time and derive an estimate of personnel needed to sustain the chosen protection level, with attention to feasibility.

4. **Requirement 4.** We will perform sensitivity and scenario analyses by varying key parameters (e.g., resource budgets, risk weights) and reporting how protection metrics and recommended deployments change.

5. **Requirement 5.** We will summarize model strengths, limitations, and key assumptions in a dedicated subsection.

6. **Requirement 6.** We will select two other reserves on different continents and outline which model components stay fixed, which inputs must be updated, what data would be needed, and how outcomes would be expected to differ.

7. **Requirement 7.** We will prepare a two-page non-technical letter to the IMMC summarizing our approach and recommendations.

2 Assumptions and Explanations

Assumption 1: **Spatial data from OpenStreetMap (via the Overpass API) is sufficient to represent park boundaries, roads, gates, waterholes, camps, terrain, and fences at a discrete grid resolution.**
Explanation: We discretize the park into grid cells (e.g., approximately 4 km $\times$ 4 km) whose centers lie inside the park boundary. Distances from each cell to the nearest road, gate, camp, and waterhole, and a binary indicator for whether the cell lies in Etosha Pan, are computed from processed OSM layers. We assume these layers are complete enough for risk and allocation; missing or outdated features may bias priorities and are noted in the limitations.

Assumption 2: **Resource allocation is strategic: each resource type (patrol, fence, monitoring) has a fixed effectiveness weight for each dominant risk type, and we allocate under budget constraints to maximize weighted protection.**
Explanation: We use a risk–resource weight matrix w_{rk} (e.g., fence weighted highly for H, monitoring for C). Allocation is performed by a greedy rule: cells are processed in order of priority, and each receives the resource that is most effective for its dominant risk among those with remaining budget. We do not model costs, travel times, or day-to-day scheduling explicitly.

Assumption 3: **Protection and threat evolve over time according to a deterministic ODE; effort (staffing and technology) is summarized as a single scalar E.**
Explanation: We use a protection–threat–effort model inspired by predatorprey dynamics: $dP/dt = r_P(1-P) - \alpha TP + \beta E$, $dT/dt = \gamma T(1-P) - \delta ET$. The initial $P(0)$ is taken from the current grid (e.g., fraction of cells protected). Effort E is in "equivalent staff" units (e.g., $E = 1$ corresponds to approximately 295 personnel). Stochastic shocks and seasonal variation are not modeled.

Assumption 4: **Reference data (park area, road length, waterhole count, staff count, fence perimeter) and ODE parameters are used as given or illustrative inputs.**
Explanation: We use IMMC-provided approximate values (22,935 km^2 park area, 3,551 km roads, 86 waterholes, $\sim$295 staff, $\sim$850 km fence) and choose ODE parameters (r_P, α, β, γ, δ) for illustration. These are not calibrated to historical incident or patrol data; staffing estimates (e.g., E^* for a target $\bar{P}$) are therefore indicative until local calibration is done.

Assumption 5: **When adapting the approach to other reserves, the model structure (grid, four risk dimensions, allocation rule, ODE) remains unchanged; only boundaries, infrastructure, threat types, and species priorities are updated.**
Explanation: We demonstrate adaptation for Yellowstone and the Sundarbans. The same equations and workflow apply; inputs (boundaries,

roads or waterways, gates, species weights, threat parameters) are replaced with site-specific data. Protection metrics and recommended deployments are then interpreted in the new context.

3 Notations

Key notations used in this paper are listed in Table 1.

Table 1: Notations used in this paper

Symbol	Description	Units
$\mathcal{G}$	Grid cells in the park	—
i	Cell index	—
$d_{\text{road}}(i)$	Distance from cell i to nearest road	km
$d_{\text{gate}}(i)$	Distance from cell i to nearest gate	km
$d_{\text{camp}}(i)$	Distance from cell i to nearest camp	km
$d_{\text{water}}(i)$	Distance from cell i to nearest waterhole	km
$r_{\text{human}}(i)$	Human-induced risk score at cell i	—
$r_{\text{nat}}(i)$	Natural threat score at cell i	—
$r_{\text{species}}(i)$	Species/habitat vulnerability score at cell i	—
$r_{\text{coverage}}(i)$	Coverage-gap score at cell i	—
$r(i)$	Dominant risk type at cell i (H, N, S, C)	—
$\text{Risk}(i)$	Combined risk at cell i	—
$\text{Protection}(i)$	Protection level at cell i ($= 1 - \text{Risk}(i)$)	—
τ	Risk threshold; protected if $\text{Risk}(i) \leq \tau$	—
$\text{Priority}(i)$	Priority score for allocation order	—
k	Resource type index (patrol, fence, monitoring)	—
w_{rk}	Weight of resource k for risk type r	—
x_{ik}	Amount of resource k in cell i	—
B_k	Budget for resource k	—
$P(t)$	Park-level protection at time t	—
$T(t)$	Threat pressure at time t	—
E	Effort ($E = 1 \Leftrightarrow \sim$295 staff)	—
r_P	ODE: protection recovery rate	—
α	ODE: threat erosion of protection	—
β	ODE: effort boost to protection	—
γ	ODE: threat growth when protection low	—
δ	ODE: effort suppression of threat	—
$\bar{P}$	Target protection level	—
E^*	Steady-state effort for $\bar{P}$	—

Notation: Symbols not listed above are defined at their first occurrence in the text.

4 Data Acquisition and Processing

4.1 Data Source and Acquisition

All spatial and attribute data for Etosha National Park are obtained from OpenStreetMap (OSM) [1]. We use the **Overpass API** [2] to query OSM within a bounding box covering the park and adjacent reserves (Etosha NP, Etosha Pan, Hobatere, Onguma). The API returns nodes, ways, and relations with their tags; the raw extract is saved as a single OSM XML file (`map.osm`). This provides a consistent, openly available base for boundaries, infrastructure, and habitat features without relying on proprietary datasets.

4.2 Indicator Selection

Indicators are chosen to support the IMMC problem: defining protection, prioritizing areas, and allocating resources. They align with the contest background (road networks, waterholes, gates, camps, salt pans, fence perimeter) and with the risk dimensions used in Task 1.

- **Boundaries and protected areas**: Park boundary, Etosha Pan polygon, and adjoining reserves (Hobatere, Onguma). Used to define the modeling domain and to classify cells as inside or outside the salt pan (natural threat and habitat vulnerability).
- **Roads**: Highway network (primary, secondary, tertiary, track, path, etc.). Used for distance-to-access (human-induced threat) and for coverage (patrol reach); total length is compared to the reference 3,551 km where available.
- **Gates**: Official entry points (e.g., Anderson, Von Lindequist, Galton, Nehale lya Mpingana). Proximity to gates proxies exposure to legal and illegal human access.
- **Waterholes**: Natural and artificial water points (nominally $\sim$86). Wildlife aggregates near water; these locations are both high-value (species vulnerability) and relevant for patrol and monitoring.
- **Camps and facilities**: Main camps (Okaukuejo, Halali, Namutoni, etc.), fuel, toilets, picnic sites, airstrips. Used as deployment bases and to measure patrol coverage (distance to nearest camp or facility).
- **Terrain and habitat**: Salt pan extent, water bodies, wetland, grassland. Used to weight natural and species-related risk (e.g., pan vs. non-pan).
- **Fences and restricted areas**: Fence lines and access-restricted zones. Used to contextualize boundary management and to check consistency with the stated fence perimeter ($\sim$850 km).

4.3 Processing Methods

From the OSM extract we (i) *filter* elements by tag keys and values (e.g., `boundary`, `highway`, `natural`, `barrier`, `tourism`, `amenity`) to obtain the layers above; (ii) *resolve geometry* by joining ways and relations to their node sequences and coordinates so that each layer has point or polyline/polygon geometry; (iii) *aggregate* point-like features (gates, waterholes, camps) to representative coordinates when they are stored as ways. For the grid-based model in Task 1 we then compute, for each cell, the mini-

mum distance to the nearest road segment, gate, camp/facility, and waterhole (in km), and a binary indicator for whether the cell centroid lies inside the Etosha Pan polygon. All distance and point-in-polygon calculations use the WGS84 coordinates from OSM, with appropriate projection or spherical approximations for the park scale.

4.4 Base Map and Outputs

Figure 1 shows the base map of the study area: park boundary, Etosha Pan, roads, gates, waterholes, camps, and related infrastructure derived from the processed OSM data. This visualization summarizes the acquired and processed layers used in the subsequent risk and protection analysis.

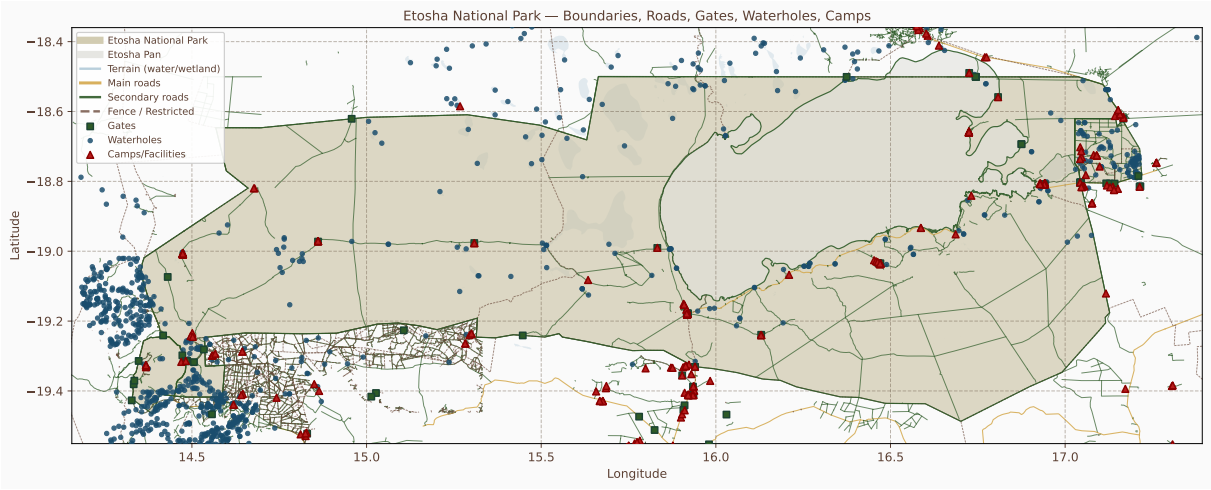


Figure 1: Base map of Etosha National Park and adjacent reserves: boundary, Etosha Pan, roads, gates, waterholes, and camps/facilities from processed OSM data.

5 Task 1: Priorities and Defining Protection

5.1 Spatial Discretization and Data Sources

We discretize the park into a regular grid of cells (approximately 4 km $\times$ 4 km per cell) whose centers lie inside the park boundary, in line with grid-based spatial conservation prioritization [4]. The boundary and habitat geometry are obtained from OpenStreetMap (OSM) data: Etosha National Park (including Hobatere and Onguma), and Etosha Pan. For each grid cell $i \in \mathcal{G}$, we compute distances to the nearest road, gate, camp/facility, and waterhole using the OSM-derived layers (roads, gates, camps, waterholes). We also assign a binary indicator for whether the cell lies inside Etosha Pan (salt pan), which is used to weight natural and habitat-related risk. The reference data (park area 22,935 km^2, 3,551 km roads, 86 waterholes, fence perimeter $\sim$850 km) are used as approximate inputs consistent with the IMMC background document [3].

5.2 Four Risk Dimensions

We structure conservation challenges along four risk dimensions, each yielding a normalized score in $[0, 1]$ for every cell i.

1. Human-induced threat $r_{\text{human}}(i)$: Risk is higher where access is easier. We use distance to roads and gates,

$$r_{\text{human}}(i) = \frac{0.6}{1 + d_{\text{road}}(i)/8} + \frac{0.4}{1 + d_{\text{gate}}(i)/15}, \tag{1}$$

with $d_{\text{road}}(i)$, $d_{\text{gate}}(i)$ in km (capped at 50 km). Closer to roads and gates implies higher exposure to poaching and unauthorized entry.

2. Natural threat $r_{\text{nat}}(i)$: The salt pan and surrounding habitat are more sensitive to fire and drought. We set

$$r_{\text{nat}}(i) = 0.2 + 0.8 \cdot \mathbb{1}[\text{cell } i \text{ in Etosha Pan}], \tag{2}$$

so that $r_{\text{nat}} \in [0.2, 1]$ and is normalized to $[0, 1]$ for comparability.

3. Species/habitat vulnerability $r_{\text{species}}(i)$: Higher in the pan and near waterholes (wildlife aggregation):

$$r_{\text{species}}(i) = \big(0.7 \cdot \mathbb{1}[\text{in pan}] + 0.4 \cdot \mathbb{1}[\text{not in pan}]\big) + \frac{0.3}{1 + d_{\text{water}}(i)/12}. \tag{3}$$

4. Coverage gap $r_{\text{coverage}}(i)$: Areas far from roads and camps are harder to patrol or monitor,

$$r_{\text{coverage}}(i) = \frac{1}{1 + \min\big(d_{\text{road}}(i), d_{\text{camp}}(i)\big)/10}. \tag{4}$$

All four scores are normalized to $[0, 1]$. The *dominant risk type* for cell i is the index (H, N, S, or C) that attains the maximum of $\{r_{\text{human}}, r_{\text{nat}}, r_{\text{species}}, r_{\text{coverage}}\}$. The *priority* used for resource allocation is

$$\text{Priority}(i) = \big(r_{\text{human}}(i) + r_{\text{nat}}(i) + r_{\text{species}}(i)\big) \cdot \big(1 + 0.5 \cdot r_{\text{coverage}}(i)\big). \tag{5}$$

5.3 Definition of Protection

We define a *combined risk* and a *protection level* so that "protection" is quantitatively measurable.

Definition 5.1 (Combined risk and protection level) *For each cell i,*

$$\text{Risk}(i) = \frac{1}{3}\big(r_{\text{human}}(i) + r_{\text{nat}}(i) + r_{\text{species}}(i)\big), \qquad \text{Protection}(i) = 1 - \text{Risk}(i). \tag{6}$$

Given a management-defined risk threshold $\tau \in [0, 1]$, we say that cell i is protected *if* $\text{Risk}(i) \le \tau$.

Thus, wildlife in the park is "protected" in a cell if the combined risk does not exceed τ; the definition may be tightened or relaxed by region or species by using different weights or thresholds in future extensions.

5.4 Results and Maps

The above procedure was implemented in Python using OSM boundaries and the Code/output CSV layers. The park is covered by 1,272 grid cells. With $\tau = 0.6$, 720 cells are classified as protected and 552 as not protected. The dominant risk types are distributed across the park: many cells are classified as coverage gap (C) or human-induced (H), with natural (N) and species/habitat (S) dominant in pan and waterhole-proximate areas.

Figure 2 shows the spatial distribution of the dominant risk type (H/N/S/C) per grid. Figure 3 shows the binary protection status (green = protected, gray = not protected) under the chosen threshold $\tau = 0.6$. These maps provide a quantitative basis for prioritizing conservation efforts and for the resource-allocation model in Task 2.

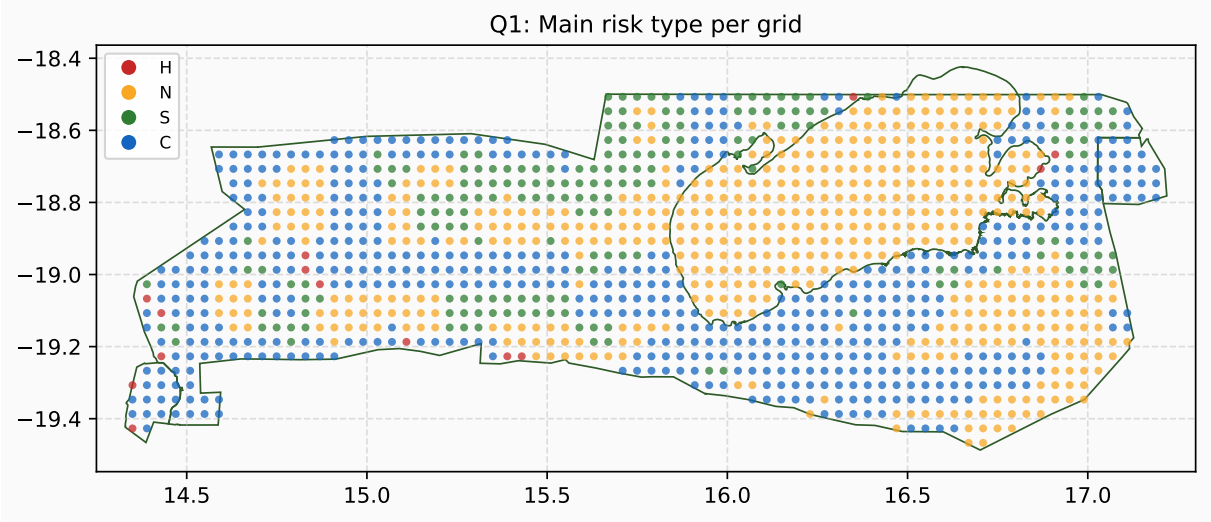


Figure 2: Dominant risk type per grid (H: human intrusion, N: natural disaster, S: species/habitat vulnerability, C: coverage gap).

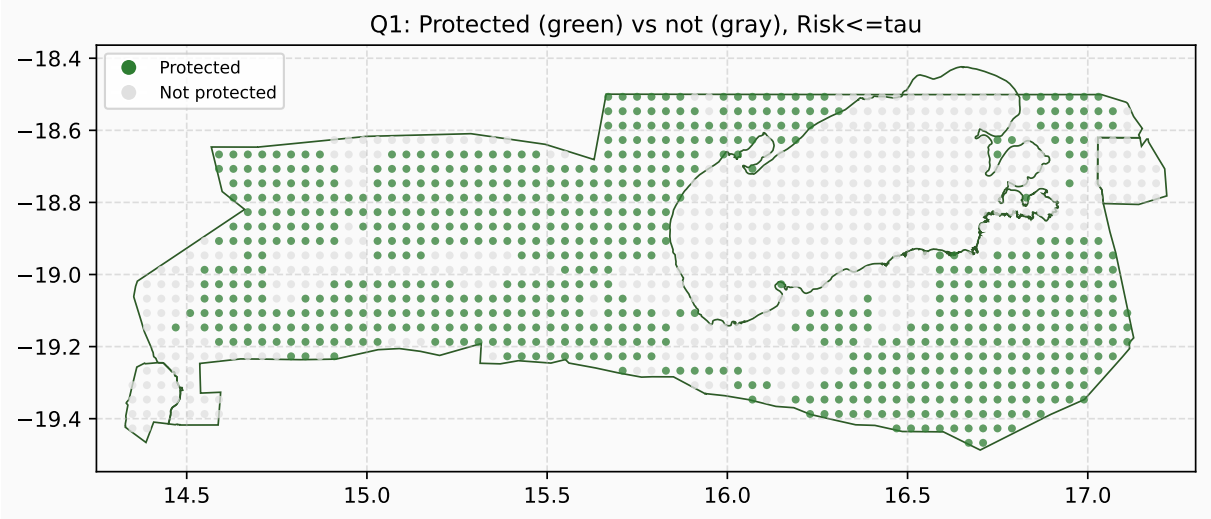


Figure 3: Protection status under threshold $\tau = 0.6$: green = protected ($\mathrm{Risk}(i) \leq \tau$), gray = not protected.

6 Task 2: Resource Allocation Model

We build a mathematical model to allocate protection resources (existing plus team-identified resources such as ranger patrols, fencing, and monitoring technology) across the park, under limited budgets, geographic constraints, and uncertainty. Optimal allocation of limited patrol and enforcement effort over space is a well-studied problem in conservation [5]; we adapt this idea to our grid and risk structure. The approach is *strategic*: match resource types to the dominant risk in each grid rather than spreading resources uniformly.

6.1 Grid and Risk Inputs from Task 1

The same spatial grid $\mathcal{G}$ as in Task 1 is used: cell centers lie inside the park boundary, and each cell i has

- four normalized risk scores $r_{\text{human}}(i)$, $r_{\text{nat}}(i)$, $r_{\text{species}}(i)$, $r_{\text{coverage}}(i)$;
- a *dominant risk type* $r(i) \in \{\text{H}, \text{N}, \text{S}, \text{C}\}$ (human intrusion, natural disaster, species vulnerability, coverage gap);
- a *priority* $\text{Priority}(i)$ for ranking cells.

These come directly from the risk model in Task 1 and drive which resource is most effective in each cell.

6.2 Resource Types and Risk–Resource Weights

We consider three aggregated resource types that align with the contest and data: **patrol** (ranger time, road-based reach), **fence** (physical barriers, gate control), and **monitoring** (remote sensing, cameras, early warning). Their effectiveness depends on the dominant risk:

- For **H** (human intrusion): fence is most effective; patrol and monitoring are also useful.
- For **N** (natural disaster): monitoring and rapid response (captured here as monitoring/patrol) are emphasized; fence is less relevant.
- For **S** (species/habitat vulnerability): patrol and monitoring support sustained presence and surveillance.
- For **C** (coverage gap): monitoring is most effective where ground access is limited; patrol is constrained by remoteness.

We encode this in a weight matrix $w_{rk} \geq 0$: the effectiveness of resource k when the dominant risk is r. For example, $w_{\text{H,fence}} > w_{\text{H,patrol}}$, $w_{\text{N,monitoring}}$ and $w_{\text{N,patrol}}$ high, $w_{\text{C,monitoring}}$ high. Weights are normalized so that for each r, $\sum_k w_{rk}$ is comparable across risk types.

6.3 Allocation Model

Decision variables: $x_{ik} \geq 0$ = amount of resource k allocated to grid i (in common units, e.g., patrol person-hours, fence length, monitoring device-hours).

Objective: Maximize effective protection by matching resources to dominant risk

and prioritizing high-priority cells:

$$\max \sum_{i \in \mathcal{G}} \text{Priority}(i) \cdot \sum_{k} w_{r(i),k}\, x_{ik}. \tag{7}$$

Thus we first identify the core risk $r(i)$ in each cell, then weight the allocation x_{ik} by $w_{r(i),k}$ so that the most effective resource for that risk contributes most.

Constraints:

- **Total resource**: For each k, $\sum_{i \in \mathcal{G}} x_{ik} \leq B_k$ (budget B_k for patrol, fence, monitoring).
- **Geography**: Allocation can be restricted by reach (e.g., patrol only along roads or within a radius of camps); monitoring can cover remote cells. We implement this via eligibility rules or capacity limits per cell/region.
- **Non-negativity**: $x_{ik} \geq 0$.

Uncertainty in risk (e.g., in r_{human} or r_{nat}) can be handled by robust or stochastic variants (e.g., worst-case or scenario-based constraints), linking to sensitivity analysis in later tasks.

6.4 Implementation and Results

We solve a simplified allocation: given budgets B_{patrol}, B_{fence}, $B_{\text{monitoring}}$, we assign resources to cells in order of decreasing $\text{Priority}(i)$. For each cell, we allocate the resource k that has the highest weight $w_{r(i),k}$ among those with remaining budget, so that high-priority cells receive the resource best matched to their dominant risk. This greedy rule yields a feasible allocation that emphasizes "identify core risk $\rightarrow$ match resource." With illustrative budgets (e.g., patrol = 120, fence = 80, monitoring = 60 units), a subset of cells (e.g., 260 grids) receive at least one resource. Cells with dominant risk H tend to receive more fence; N and C more monitoring; S more patrol, consistent with the risk–resource weights.

Figure 4 shows the spatial distribution of the allocation: color indicates dominant risk type (H/N/S/C) and symbol or style indicates the primary resource assigned (patrol, fence, monitoring). The map supports strategic discussion of where to reinforce fencing, patrol routes, or monitoring coverage.

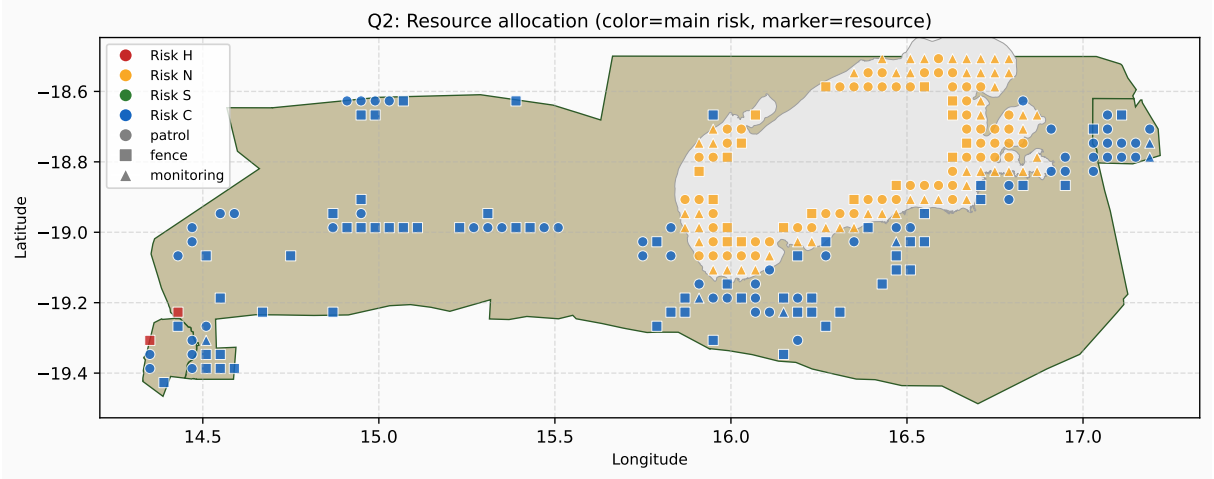


Figure 4: Resource allocation by grid: color = dominant risk type (H/N/S/C), symbol = primary resource (patrol, fence, monitoring).

7 Task 3: Protection over Time and Staffing Need

We use the modeling approach to analyze how well the park is protected over time and to estimate the number of human personnel required to maintain a chosen level of protection. The analysis builds on a *protection–threat–effort* dynamical system inspired by predator–prey (Lotka–Volterra) ideas: protection level and threat pressure evolve over time, with staffing (and technology) as a control that suppresses threat and supports protection.

7.1 Link to Tasks 1 and 2

The same spatial grid $\mathcal{G}$ and risk-based definition of protection from Task 1 are used. We define a *park-level protection level* $P(t) \in [0, 1]$ as the fraction of grid cells that are protected ($\mathrm{Risk}(i) \leq \tau$) at time t, or as the mean protection level $\frac{1}{|\mathcal{G}|}\sum_i \mathrm{Protection}(i)$. The initial value $P(0)$ is set from the current Task 1 output (e.g., the fraction of cells classified as protected). The resource allocation in Task 2 is summarized as a total *effort* E (e.g., equivalent person-years of patrol, fence, and monitoring), so that $E = 1$ corresponds to the reference staffing of approximately 295 personnel.

7.2 Protection–Threat–Effort ODE

Let $P(t)$ denote the park-level protection level and $T(t) \geq 0$ a threat pressure index (poaching opportunity, fire risk, etc.). Effort E is treated as constant or piecewise constant (e.g., fixed staffing over a planning horizon). The dynamics are

$$\frac{dP}{dt} = r_P(1 - P) - \alpha TP + \beta E, \qquad \frac{dT}{dt} = \gamma T(1 - P) - \delta ET. \tag{8}$$

Here r_P is the natural recovery rate of protection; α is the erosion of protection by threat; β is the rate at which effort increases protection; γ is the growth of threat when protection is low; and δ is the suppression of threat by effort. In analogy to Lotka–Volterra [6, 7]: P plays the role of "prey" (protection is consumed by threat), T the role of "predator" (threat grows when protection is weak), and E is an external control that both boosts P and suppresses T, allowing the system to reach a desired equilibrium.

7.3 Steady-State Staffing Need

At equilibrium ($dP/dt = dT/dt = 0$), for a target protection level $\bar{P}$, the required effort satisfies $\gamma(1 - \bar{P}) = \delta E^*$, so

$$E^* = \frac{\gamma}{\delta}(1 - \bar{P}). \tag{9}$$

Staffing in headcount is then approximated as $295 \times E^*$ when $E = 1$ corresponds to 295 personnel. If the computed E^* exceeds the current capacity, management may need to raise efficiency (e.g., technology, route optimization), accept a lower target $\bar{P}$, or phase in improvements over time.

7.4 Numerical Results and Figures

The ODE is solved numerically using the initial $P(0)$ and $T(0)$ derived from `grid_risk.csv`. Figure 5 shows $P(t)$ and $T(t)$ under different fixed effort levels E. When E is low, protection declines and threat rises; when E is sufficient, protection stabilizes and threat is controlled. Figure 6 shows the steady-state effort $E^*(\bar{P})$ and the corresponding estimated personnel needed to maintain a target protection level $\bar{P}$.

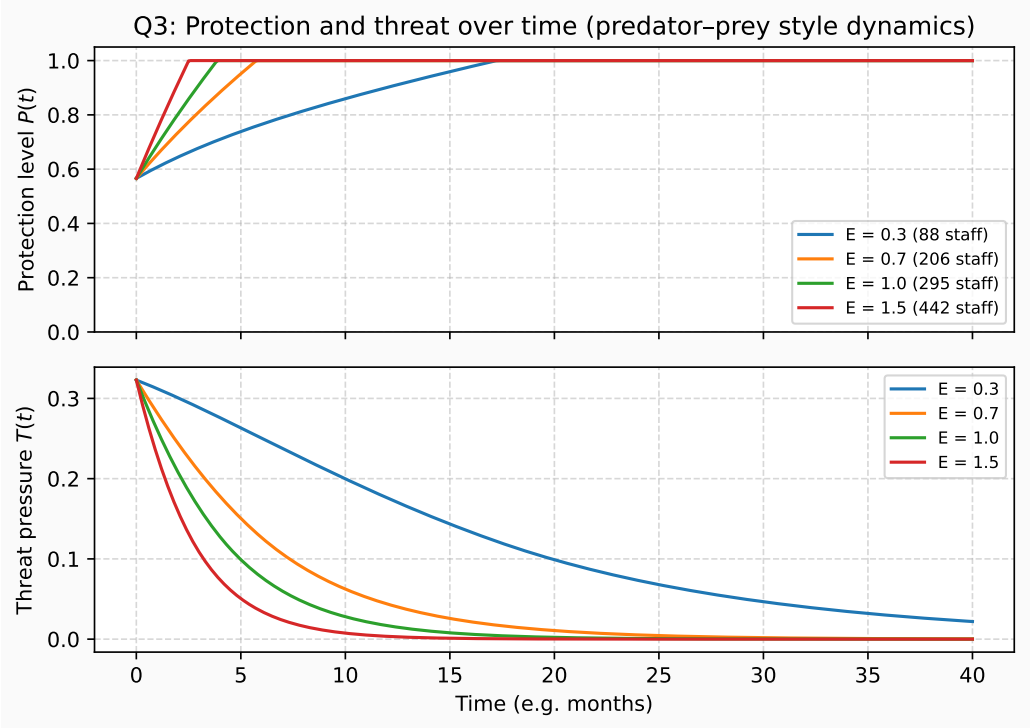


Figure 5: Protection level $P(t)$ and threat pressure $T(t)$ over time under different effort levels E (effort 1 corresponds to ∼295 staff).

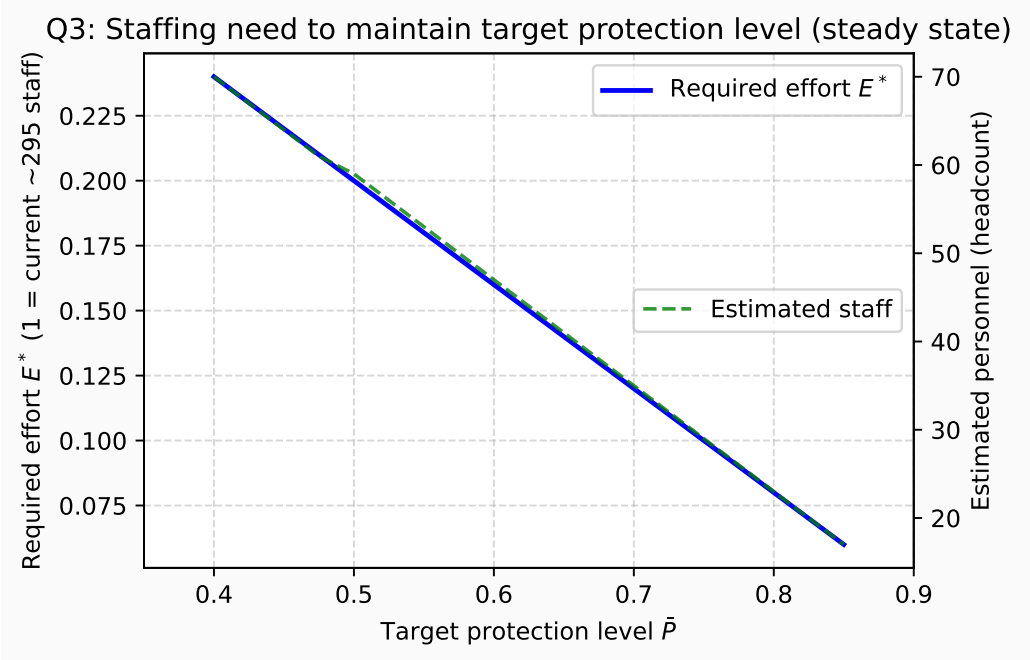


Figure 6: Steady-state effort E^* and estimated personnel required to maintain target protection level $\bar{P}$.

With the default parameters in the implementation, maintaining a 60% protection level in steady state requires an effort $E^* \approx 0.16$, corresponding to roughly 47 personnel under the linear scaling. Parameter calibration (e.g., so that $E^* = 1$ for a chosen $\bar{P}$) can align the model with management targets and link directly to resource use, feasibility, and practicality for discussion in the sensitivity analysis (Task 4).

8 Task 4: Sensitivity and Scenario Analysis

8.1 Sensitivity Analysis: Parameters and Required Effort

The steady-state effort E^* required to maintain a target protection level $\bar{P} = 0.6$ depends on the ODE parameters. In our formulation $E^* = (\gamma/\delta)(1 - \bar{P})$, so E^* is sensitive to γ (threat growth when protection is low) and δ (suppression of threat by effort), and is unaffected by r_P and α in the steady-state formula. To illustrate how assumptions affect the strategy, we vary each of r_P, α, γ, and δ by scale factors 0.8, 1.0, and 1.2 (others fixed at baseline) and compute E^* and the corresponding estimated staff ($295 \times E^*$).

Figure 7 shows the sensitivity of E^* to these parameters. When γ increases (threat grows faster when protection is weak), more effort is needed; when δ increases (effort suppresses threat more effectively), less effort is needed. Parameters r_P and α do not appear in E^* for this equilibrium, so the strategy is robust to their misspecification in terms of steady-state staffing; they nevertheless affect the transient dynamics of $P(t)$ and $T(t)$. If E^* is sensitive to a parameter, that parameter should be estimated or calibrated carefully; where sensitivity is low, the strategy is more robust.

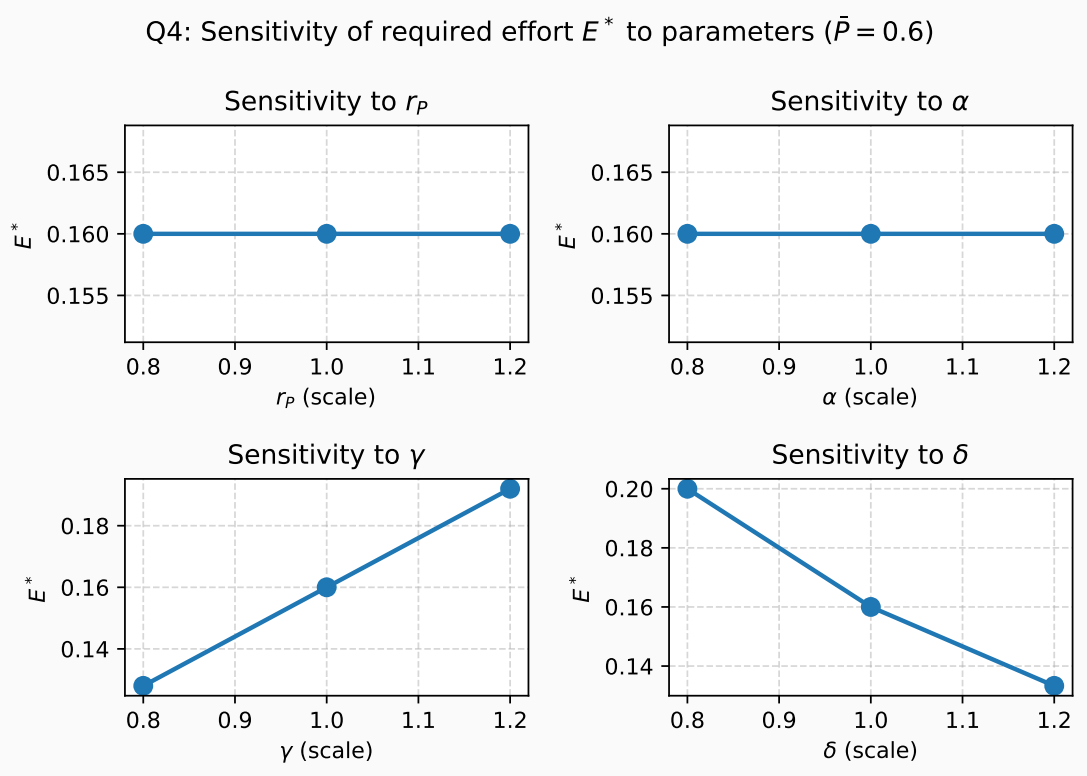


Figure 7: Sensitivity of required effort E^* (for $\bar{P} = 0.6$) to ODE parameters; each parameter is scaled by 0.8, 1.0, 1.2.

8.2 Scenario Analysis: Reduced Personnel and Effective Effort

We model changes in available resources by varying the effective effort E: reduced personnel or fewer technological assets are represented by a lower E (e.g., $E = 0.7$ for roughly 30% fewer staff, $E = 0.5$ for 50% fewer). Altered patrol schedules can be interpreted as a change in effective coverage or response, also captured here by a reduced E. For each scenario we solve the protection–threat ODE from Task 3 and compare $P(t)$ over time.

Figure 8 shows protection level $P(t)$ under baseline effort ($E = 1$, ∼295 staff), $E = 0.7$ (∼207 staff, -30%), and $E = 0.5$ (∼148 staff, -50%). With less effort, protection declines and settles to a lower equilibrium; the time to reach the new equilibrium and the final level inform how critical it is to maintain staffing or to compensate with technology. Strategy implications: if a cut in personnel or technology is anticipated, the model quantifies the drop in protection and supports the case for a minimum staffing level or for technology substitution to keep effective E from falling below a chosen threshold.

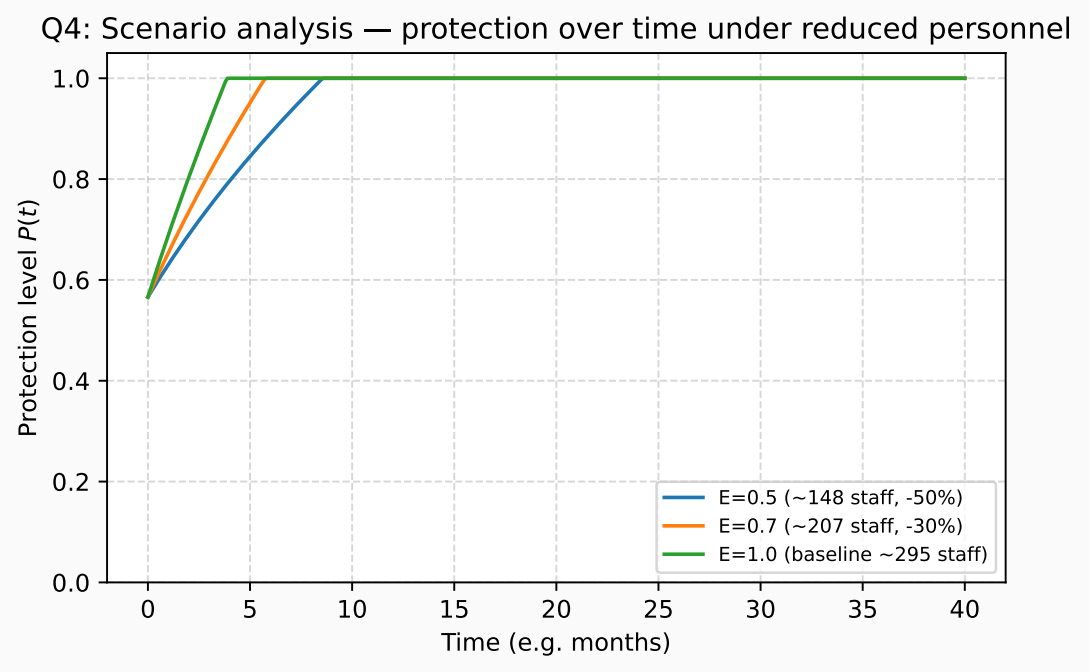


Figure 8: Protection level $P(t)$ over time under different effort scenarios: baseline ($E = 1$), -30% staff ($E = 0.7$), -50% staff ($E = 0.5$).

9 Task 5: Model Strengths and Limitations

We discuss the strengths and limitations of our modeling approach and the assumptions that may restrict its applicability.

9.1 Strengths

Quantitative and measurable definition of protection. Protection is defined in terms of a combined risk index and a threshold τ, so that every grid cell (and the park

as a whole) has a clear protected/not-protected status. This supports reporting, target-setting, and comparison over time or across scenarios.

Structured risk framework. The four risk dimensions (human intrusion, natural threat, species/habitat vulnerability, coverage gap) and the dominant-risk classification (H/N/S/C) link geography (roads, gates, waterholes, camps, salt pan) to conservation priorities in a transparent way. The same grid and risk outputs feed both the allocation model (Task 2) and the dynamics (Task 3).

Strategic resource allocation. Rather than spreading resources uniformly, the model matches resource types (patrol, fence, monitoring) to the dominant risk in each cell via a risk–resource weight matrix. This aligns with the contest emphasis on strategic deployment under limited resources.

Dynamics and staffing. The protection–threat–effort ODE provides a simple, interpretable description of how protection and threat evolve over time and how effort (staffing and technology) affects equilibrium. It yields an explicit formula for the effort required to maintain a target protection level and connects to scenario analysis (e.g., reduced personnel).

Sensitivity and scenarios. Parameter sensitivity and resource scenarios are analyzed numerically, so that the influence of assumptions and of changes in personnel or technology on protection is quantified and illustrated with figures.

Open, reproducible data. The use of OpenStreetMap and the Overpass API makes the spatial base (boundaries, roads, gates, waterholes, camps) openly available and reproducible for other reserves.

9.2 Limitations and Restrictive Assumptions

Spatial aggregation. Park-level protection $P(t)$ is a single aggregate (e.g., fraction of cells protected). Spatial heterogeneity (e.g., one region improving while another deteriorates) is not represented in the dynamics, which may understate local crises or overstate overall protection if threats are concentrated.

Effort as a scalar. Total effort E is one number (e.g., equivalent staff). The mix of patrol, fence, and monitoring and their geographic distribution are treated in Task 2 but not explicitly coupled to the ODE in Task 3; the dynamics do not distinguish, for example, technology-only versus patrol-only changes.

Deterministic dynamics. The ODE is deterministic. Stochastic shocks (e.g., a single severe poaching event or fire) or seasonal variation in threat or effort are not modeled, so the approach is better suited to medium-term planning than to short-term incident response.

Parameter calibration. The ODE parameters (r_P, α, β, γ, δ) and the risk formulas (distance decay, pan indicator) are chosen for illustration and consistency with the reference data. They are not calibrated to historical protection or incident data; estimates of required staff (e.g., $295 \times E^*$) are therefore indicative rather than validated.

Data completeness. OSM may underrepresent or misplace some features (e.g., waterholes, fences, or restricted areas). The grid and risk scores depend on this input; missing or outdated data can bias priorities and allocation.

No explicit cost or feasibility constraints. The allocation model maximizes a weighted protection objective subject to resource budgets but does not include costs, travel times,

or operational constraints (e.g., shift patterns, equipment availability). The staffing discussion is in headcount or effort units, not in a detailed cost analysis.

9.3 Applicability

The approach is best suited to strategic, park-level planning: defining protection, prioritizing areas, allocating resources by risk type, and assessing how protection and staffing need respond to parameter and resource changes. It is less suited to tactical, day-to-day deployment, to species-specific or habitat-specific targets without further disaggregation, or to reserves where high-resolution threat or patrol data would allow a more data-driven calibration. Extending the model to other parks (Task 6) would require updating boundaries, infrastructure, and threat types while keeping the same structure (grid, risk dimensions, allocation rule, and ODE).

10 Task 6: Adaptation to Other Protected Areas

We demonstrate how the modeling approach can be adapted to two other large, well-known protected areas on different continents: **Yellowstone National Park** (USA, North America) and the **Sundarbans** (India/Bangladesh, Asia). Both are UNESCO-related, span thousands of square kilometers, and face multiple conservation threats with limited resources.

10.1 Choice of Sites

Yellowstone National Park (Wyoming, Montana, Idaho, USA) covers approximately 8,991 km^2 of mountains, forests, and geothermal features. It supports iconic species (bison, grizzly bear, wolf, elk) and experiences high visitor use, wildlife-vehicle conflict, and wildfire risk. Roads and gates structure access; patrols and monitoring are central to management.

Sundarbans (India and Bangladesh) is the world's largest contiguous mangrove forest (about 10,000 km^2 in the Indian part alone), a UNESCO World Heritage Site. It is critical for Bengal tigers, estuarine crocodiles, and many other species. Threats include poaching, human–tiger conflict, cyclones, and sea-level rise. Terrain is largely wetland and tidal; mobility relies on boats and limited trails rather than a road network comparable to Etosha or Yellowstone.

10.2 Components That Remain Unchanged

The *model structure* is reused without change:

- Spatial discretization into a grid of cells (with cell size adapted to park scale).
- The four risk dimensions (human intrusion, natural threat, species/habitat vulnerability, coverage gap) and the rule for dominant risk type (H/N/S/C).
- The definition of protection via a combined risk index and a threshold τ, and of park-level $P(t)$ as the fraction of cells protected or mean protection level.

- The resource-allocation objective: maximize weighted protection by matching resource type (patrol, fence, monitoring) to dominant risk under budget constraints.
- The protection–threat–effort ODE and the formula $E^* = (\gamma/\delta)(1 - \bar{P})$ for steady-state staffing need.
- The sensitivity and scenario framework (parameter variation, effort scenarios).

Data acquisition via Overpass API and OSM remains the same in principle; only the bounding box and tag filters change.

10.3 Inputs and Parameters To Modify

Geography and mobility. Replace Etosha's boundary, Etosha Pan, and reserves (Hobatere, Onguma) with the new park boundaries (e.g., Yellowstone polygon; Sundarbans forest and buffer). For Sundarbans, "roads" become waterways or designated boat routes where relevant; distance-to-road may be replaced or supplemented by distance-to-patrol route or to guard posts. Terrain layers (salt pan in Etosha) are replaced by local features (e.g., geothermal zones in Yellowstone, tidal zones or core vs. buffer in Sundarbans).

Access points and infrastructure. Use OSM (or local GIS) for gates, entrances, visitor centers, and ranger posts. Yellowstone has a well-mapped road and gate network; Sundarbans has fewer roads and more boat landings and watchtowers. Camps and facilities are redefined accordingly.

Threat types and species priorities. Human intrusion: poaching and illegal entry (all three sites); add wildlife-vehicle conflict and visitor pressure in Yellowstone. Natural threats: wildfire and drought (Etosha, Yellowstone); cyclones and sea-level rise (Sundarbans). Species/habitat vulnerability: weight by local flagship species (e.g., rhino in Etosha, bison/wolf in Yellowstone, tiger in Sundarbans) and habitat sensitivity. Coverage gap: in Sundarbans, many areas are accessible mainly by boat, so "distance to patrol route" or "distance to guard post" drives coverage risk more than roads.

Detection and technology. If one site has better coverage of camera traps, drones, or satellite monitoring, the risk–resource weights (e.g., higher weight for monitoring in coverage-gap-dominated cells) or the effective effort E can be adjusted to reflect that capability. No change to the model equations is required; only the interpretation of "monitoring" and its budget/effectiveness.

10.4 Data Required for Calibration

Common to both. Boundaries and infrastructure from OSM (or official GIS): park polygon(s), roads or waterways, gates/entrances, ranger posts, visitor facilities. Terrain or land-cover (e.g., forest, wetland, mangrove) for habitat weighting. Where available: road lengths, number of gates, approximate staff or patrol presence, to set scale (e.g., reference $E = 1$).

Yellowstone. Road network and entrance stations; wildfire history or hazard maps; visitor statistics; species distribution or priority areas (e.g., bison, wolf) for species vulnerability; data on wildlife-vehicle incidents if used to weight human-related risk near roads.

Sundarbans. Mangrove and water-body geometry; boat routes or patrol corridors; locations of guard posts, watchtowers, and buffer-zone settlements; cyclone exposure or flood maps; tiger conflict or poaching incident reports (if available) to weight threats and priorities.

10.5 Expected Shift in Protection Metrics and Resource Deployment

Yellowstone. With a dense road network and many visitors, human-related and coverage-gap risks may be significant near roads and gates; natural (fire) risk is high in large forested areas. We would expect a larger share of cells classified as H or C than in Etosha, and a need for strong monitoring and patrol along corridors. Fence is less relevant than in Etosha (no single perimeter fence). Staffing and effort E would be scaled to the reported ranger and resource budget; protection level $P(t)$ and required E^* would be interpreted in the same way as for Etosha, with different baseline $P(0)$ and possibly different γ, δ if threat dynamics differ.

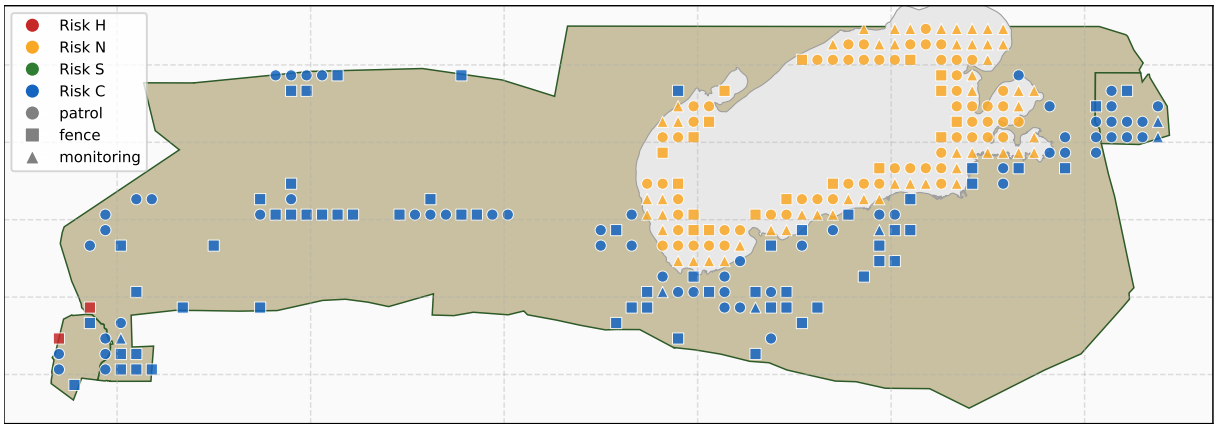
Sundarbans. Coverage gap and species (tiger) vulnerability are likely to dominate many cells; human–tiger conflict and poaching drive human-related risk. Terrain limits patrol to boats and some foot trails, so "monitoring" (e.g., camera traps, satellite) may get higher weight for coverage. We would expect more cells with dominant risk C or S, and resource allocation to emphasize monitoring and targeted patrol rather than fence. Protection metrics (fraction of cells protected, $P(t)$) would be computed the same way, but the geographic pattern of high-risk cells and the recommended mix of patrol vs. monitoring would differ from both Etosha and Yellowstone.

In both adaptations, the same outputs (risk maps, allocation maps, $P(t)$, E^*, sensitivity and scenario figures) support strategic planning; only the inputs, parameters, and interpretation are site-specific.

Letter to the IMMC

Dear IMMC,

Large wildlife reserves such as Etosha National Park face a common challenge: how to protect species and habitats across vast areas with limited staff and technology. In this letter we summarize a modeling approach that helps answer three questions in a practical, measurable way: *What do we mean by "protected"? Where should we deploy resources?* and *How many people and how much technology are needed to sustain protection over time?* We hope this summary is useful to decision-makers and that the full report provides the detail needed for implementation.



A measurable definition of protection

We divide the park into a grid and assign each cell a risk level from four sources: human intrusion (poaching, unauthorized entry), natural hazards (fire, drought), species and habitat vulnerability, and "coverage gaps" (areas that are hard to patrol or monitor). A cell is **protected** when its combined risk stays below a chosen threshold. Managers can then report, for example, that "X% of the park is protected" and set targets. This definition is explicit, quantitative, and can vary by region or species if desired.

Strategic resource allocation

Rather than spreading effort evenly, we **match** each area to the resource that best addresses its main risk. Where human intrusion dominates, physical barriers and gate control are emphasized; where the main issue is lack of visibility, remote monitoring (e.g., cameras, drones) is prioritized; where natural or species-related threats are high,

patrol and early response get more weight. Our maps show where to emphasize patrols, fencing, or monitoring so that limited resources have the largest impact. This strategic deployment is a central recommendation of our work.

Protection over time and staffing need

We use a simple dynamic model to describe how protection and threat levels evolve when staffing and technology are applied. Two results stand out. First, we can estimate the **minimum effort** (in equivalent staff and technology) required to maintain a chosen protection level. Second, we show how protection **declines** when personnel or technology are cut—for example, a 30% or 50% reduction in effective effort leads to a clear drop in protection over time. We also test how sensitive these conclusions are to our assumptions; some parameters matter more than others for planning. These results support both target-setting and the case for sustaining (or increasing) resources where protection is a priority.

Main recommendations

We recommend that managers and conservation bodies: (1) adopt a **risk-based, measurable definition** of protection for reporting and target-setting; (2) allocate resources **strategically** by risk type rather than uniformly; (3) use dynamic and scenario analysis to plan for **minimum staffing and technology** and to anticipate the impact of budget or resource cuts; and (4) when applying the approach to other reserves (we illustrate with Yellowstone and the Sundarbans), keep the same model structure and update only boundaries, infrastructure, threats, and species priorities. The approach is designed to be **reproducible** using open map data (OpenStreetMap) and can be adapted to different continents and management contexts.

Limitations

Our model is best suited to **strategic, park-level** planning rather than day-to-day operations. Staffing estimates are illustrative until calibrated with local data, and we do not include detailed cost or operational constraints. We discuss strengths and limitations at length in the full report.

Closing

We believe this approach offers a practical, transparent way to inform protection strategies for Etosha and for other large reserves. We are grateful to the IMMC for the opportunity to contribute to this important challenge and hope that our work supports better decisions for wildlife and habitats at scale.

Sincerely,
The Team # 26672879

References

[1] OpenStreetMap Contributors. *OpenStreetMap*. OpenStreetMap Foundation, 2024. openstreetmap.org.

[2] OpenStreetMap. 2024. wiki.openstreetmap.org/wiki/Overpass_API.

[3] Namibia. Ministry of Environment, Forestry and Tourism. "Etosha National Park." *MEFT*, n.d. meft.gov.na/national-parks/etosha-national-park.

[4] Lehtomäki, Joona, and Atte Moilanen. "Methods and Workflow for Spatial Conservation Prioritization Using Zonation." *Environmental Modelling & Software*, vol. 47, 2013, pp. 128–37.

[5] Moore, Jennifer F., et al. "Optimal Allocation of Law Enforcement Patrol Effort to Mitigate Poaching Activities." *Ecological Applications*, vol. 31, no. 5, 2021, e02337.

[6] Lotka, Alfred J. *Elements of Physical Biology*. Williams and Wilkins, 1925.

[7] Volterra, Vito. "Variazioni e fluttuazioni del numero d'individui in specie animali conviventi." *Memorie della Accademia Nazionale dei Lincei*, vol. 2, 1926, pp. 31–113.

[8] "Rhino Protection Intensified After Recent Poaching in Etosha National Park." *The Namibian*, 2022. namibian.org.

[9] University of Namibia Repository. "Dynamics of Namibian Defence Force Intervention in Anti-Poaching Operations of Etosha National Park (2016–2022)." repository.unam.edu.na, 2022.

[10] Scott-Hayward, L. A. S., et al. 'Automated Surface Feature Selection Using SALSA2D: Assessing Distribution of Elephant Carcasses in Etosha National Park'. arXiv [Stat.ME], 2023, arxiv.org/abs/2202.07977. arXiv.

A Code

A.1 Data Processing

Listing A.1: Data Processing

```
import os
import csv
import xml.etree.ElementTree as ET
from collections import defaultdict

CODE_DIR = os.path.dirname(os.path.abspath(__file__))
OSM_PATH = os.path.join(CODE_DIR, "map.osm")
OUT_DIR = os.path.join(CODE_DIR, "output")
BOUNDARY_IDS = {"relation": ["2982497", "5694335", "14066259"],
    ↪ "way": ["863042084", "891246359"]}

def _tag_key(t):
    for a in t.attrib:
        if a == "k" or a.endswith("}k"): return t.attrib[a]
```

```
    return ""

def _tag_value(t):
    for a in t.attrib:
        if a == "v" or a.endswith("}v"): return t.attrib[a]
    return ""

def _get_tags(elem):
    return {_tag_key(t): _tag_value(t) for t in elem.findall("tag")
        ↪ if _tag_key(t)}

def parse_osm():
    nodes, node_tags = {}, defaultdict(dict)
    ways_nds, way_tags = {}, defaultdict(dict)
    relations, relation_members = {}, defaultdict(list)
    relation_tags = defaultdict(dict)
    for event, elem in ET.iterparse(OSM_PATH, events=("end",)):
        if elem.tag == "node":
            nid, lat, lon = elem.get("id"), elem.get("lat"),
                ↪ elem.get("lon")
            if nid and lat and lon:
                nodes[nid] = (float(lat), float(lon))
                node_tags[nid] = _get_tags(elem)
            elem.clear()
        elif elem.tag == "way":
            wid = elem.get("id")
            if wid:
                ways_nds[wid] = [nd.get("ref") for nd in
                    ↪ elem.findall("nd") if nd.get("ref")]
                way_tags[wid] = _get_tags(elem)
            elem.clear()
        elif elem.tag == "relation":
            rid = elem.get("id")
            if rid:
                relation_tags[rid] = _get_tags(elem)
                for m in elem.findall("member"):
                    relation_members[rid].append({"type":
                        ↪ m.get("type"), "ref": m.get("ref"),
                        ↪ "role": m.get("role", "")})
                relations[rid] = True
            elem.clear()
    return nodes, node_tags, ways_nds, way_tags, relations,
        ↪ relation_members, relation_tags

def write_boundaries(relation_tags, way_tags):
    fieldnames = ["id", "type", "name", "boundary", "natural",
        ↪ "leisure"]
    rows = []
    for rid in BOUNDARY_IDS["relation"]:
        t = relation_tags.get(rid, {})
        rows.append({"id": rid, "type": "relation", "name":
            ↪ t.get("name", ""), "boundary": t.get("boundary", ""),
            ↪ "natural": t.get("natural", ""), "leisure": ""})
    for wid in BOUNDARY_IDS["way"]:
        t = way_tags.get(wid, {})
        rows.append({"id": wid, "type": "way", "name": t.get("name",
            ↪ ""), "boundary": t.get("boundary", ""), "natural":
            ↪ t.get("natural", ""), "leisure": t.get("leisure", "")})
    _write_csv(os.path.join(OUT_DIR, "boundaries.csv"), rows,
```

```
        ↪ fieldnames=fieldnames)

def write_roads(ways_nds, way_tags, nodes):
    highway_values = ("primary", "secondary", "tertiary", "track",
        ↪ "path", "unclassified", "residential", "service")
    rows = []
    for wid, tags in way_tags.items():
        h = tags.get("highway")
        if not h: continue
        nds = ways_nds.get(wid, [])
        lat, lon = (nodes[nds[0]] if nds and nds[0] in nodes else
            ↪ (None, None))
        rows.append({"way_id": wid, "highway": h, "name":
            ↪ tags.get("name", ""), "ref": tags.get("ref", ""),
            ↪ "surface": tags.get("surface", ""), "node_count":
            ↪ len(nds), "lat": lat, "lon": lon})
    _write_csv(os.path.join(OUT_DIR, "roads.csv"), rows)
    road_way_ids = {r["way_id"] for r in rows}
    geom_rows = []
    for wid in road_way_ids:
        for i, nid in enumerate(ways_nds.get(wid, [])):
            if nid in nodes:
                lat, lon = nodes[nid]
                geom_rows.append({"way_id": wid, "sequence": i,
                    ↪ "node_id": nid, "lat": lat, "lon": lon})
    _write_csv(os.path.join(OUT_DIR, "roads_geometry.csv"),
        ↪ geom_rows)

def write_gates(nodes, node_tags, ways_nds, way_tags):
    keywords = ("gate", "anderson", "von lindequist", "galton",
        ↪ "nehale")
    rows = []
    for nid, tags in node_tags.items():
        if nid not in nodes: continue
        name = (tags.get("name") or "").lower()
        if tags.get("barrier") == "gate" or any(k in name for k in
            ↪ keywords):
            lat, lon = nodes[nid]
            rows.append({"node_id": nid, "lat": lat, "lon": lon,
                ↪ "name": tags.get("name", ""), "element": "node"})
    for wid, tags in way_tags.items():
        name = (tags.get("name") or "").lower()
        if tags.get("barrier") == "gate" or any(k in name for k in
            ↪ keywords):
            nds = ways_nds.get(wid, [])
            lat, lon = (nodes[nds[0]] if nds and nds[0] in nodes
                ↪ else (None, None))
            rows.append({"way_id": wid, "lat": lat, "lon": lon,
                ↪ "name": tags.get("name", ""), "element": "way"})
    _write_csv(os.path.join(OUT_DIR, "gates.csv"), rows)

def write_waterholes(nodes, node_tags, ways_nds, way_tags):
    rows = []
    for nid, tags in node_tags.items():
        if nid not in nodes: continue
        nat, water, waterway = tags.get("natural"),
            ↪ tags.get("water"), tags.get("waterway")
        if nat == "waterhole" or water or waterway or (nat ==
            ↪ "water" and "pan" in (tags.get("name") or "").lower()):
```

```
105                 lat, lon = nodes[nid]
106                 rows.append({"node_id": nid, "lat": lat, "lon": lon,
                        ↪ "name": tags.get("name", ""), "natural": nat,
                        ↪ "water": water, "waterway": waterway, "element":
                        ↪ "node"})
107         for wid, tags in way_tags.items():
108             nat, water, waterway = tags.get("natural"),
                    ↪ tags.get("water"), tags.get("waterway")
109             if nat == "waterhole" or (nat == "water" and water) or
                    ↪ waterway:
110                 nds = ways_nds.get(wid, [])
111                 lat, lon = (nodes[nds[0]] if nds and nds[0] in nodes
                        ↪ else (None, None))
112                 rows.append({"way_id": wid, "lat": lat, "lon": lon,
                        ↪ "name": tags.get("name", ""), "natural": nat,
                        ↪ "water": water, "waterway": waterway, "element":
                        ↪ "way"})
113         _write_csv(os.path.join(OUT_DIR, "waterholes.csv"), rows)
114
115     def write_camps_facilities(nodes, node_tags, ways_nds, way_tags):
116         rows = []
117         for nid, tags in node_tags.items():
118             if nid not in nodes: continue
119             t, a, aw, lei = tags.get("tourism"), tags.get("amenity"),
                    ↪ tags.get("aeroway"), tags.get("leisure")
120             if t in ("camp_site", "hotel", "guest_house") or a in
                    ↪ ("fuel", "toilets") or aw or (lei and "picnic" in (lei
                    ↪ or "")):
121                 lat, lon = nodes[nid]
122                 rows.append({"node_id": nid, "lat": lat, "lon": lon,
                        ↪ "name": tags.get("name", ""), "tourism": t or "",
                        ↪ "amenity": a or "", "element": "node"})
123         for wid, tags in way_tags.items():
124             t, a, aw, lei = tags.get("tourism"), tags.get("amenity"),
                    ↪ tags.get("aeroway"), tags.get("leisure")
125             if t or a in ("fuel", "toilets") or aw or (lei and "picnic"
                    ↪ in (lei or "")):
126                 nds = ways_nds.get(wid, [])
127                 lat, lon = (nodes[nds[0]] if nds and nds[0] in nodes
                        ↪ else (None, None))
128                 rows.append({"way_id": wid, "lat": lat, "lon": lon,
                        ↪ "name": tags.get("name", ""), "tourism": t or "",
                        ↪ "amenity": a or "", "element": "way"})
129         _write_csv(os.path.join(OUT_DIR, "camps_facilities.csv"), rows)
130
131     def write_terrain(way_tags, relation_tags, ways_nds, nodes):
132         fieldnames = ["id", "type", "name", "natural", "landuse"]
133         rows = []
134         for rid in BOUNDARY_IDS["relation"]:
135             t = relation_tags.get(rid, {}); nat, landuse =
                    ↪ t.get("natural"), t.get("landuse")
136             if nat or landuse: rows.append({"id": rid, "type":
                    ↪ "relation", "name": t.get("name", ""), "natural": nat
                    ↪ or "", "landuse": landuse or ""})
137         terrain_way_ids = set()
138         for wid, tags in way_tags.items():
139             nat, landuse = tags.get("natural"), tags.get("landuse")
140             if nat in ("water", "wetland", "grassland") or landuse in
                    ↪ ("grass", "meadow"):
```

```
141             rows.append({"id": wid, "type": "way", "name":
                    ↪ tags.get("name", ""), "natural": nat or "",
                    ↪ "landuse": landuse or ""});
                    ↪ terrain_way_ids.add(wid)
142     _write_csv(os.path.join(OUT_DIR, "terrain.csv"), rows,
            ↪ fieldnames=fieldnames)
143     geom_rows = []
144     for wid in terrain_way_ids:
145         for i, nid in enumerate(ways_nds.get(wid, [])):
146             if nid in nodes: geom_rows.append({"way_id": wid,
                    ↪ "sequence": i, "node_id": nid, "lat":
                    ↪ nodes[nid][0], "lon": nodes[nid][1]})
147     if geom_rows: _write_csv(os.path.join(OUT_DIR,
            ↪ "terrain_geometry.csv"), geom_rows)
148
149 def write_fence_restricted(way_tags, node_tags, ways_nds, nodes):
150     fieldnames = ["node_id", "way_id", "barrier", "boundary",
            ↪ "access", "military", "name"]
151     rows, fence_way_ids = [], set()
152     for wid, tags in way_tags.items():
153         bar, bound, acc, mil = tags.get("barrier"),
                ↪ tags.get("boundary"), tags.get("access"),
                ↪ tags.get("military")
154         if bar == "fence" or bound or acc in ("no", "private") or
                ↪ mil:
155             rows.append({"node_id": "", "way_id": wid, "barrier":
                    ↪ bar or "", "boundary": bound or "", "access": acc
                    ↪ or "", "military": mil or "", "name":
                    ↪ tags.get("name", "")}); fence_way_ids.add(wid)
156     for nid, tags in node_tags.items():
157         if tags.get("access") in ("no", "private") or
                ↪ tags.get("military"):
158             rows.append({"node_id": nid, "way_id": "", "barrier":
                    ↪ "", "boundary": "", "access": tags.get("access",
                    ↪ ""), "military": tags.get("military", ""), "name":
                    ↪ tags.get("name", "")})
159     _write_csv(os.path.join(OUT_DIR, "fence_restricted.csv"), rows,
            ↪ fieldnames=fieldnames)
160     geom_rows = [{"way_id": wid, "sequence": i, "node_id": nid,
            ↪ "lat": nodes[nid][0], "lon": nodes[nid][1]} for wid in
            ↪ fence_way_ids for i, nid in enumerate(ways_nds.get(wid,
            ↪ [])) if nid in nodes]
161     if geom_rows: _write_csv(os.path.join(OUT_DIR,
            ↪ "fence_restricted_geometry.csv"), geom_rows)
162
163 def _write_csv(path, rows, fieldnames=None):
164     if not rows: return
165     keys = fieldnames or list(rows[0].keys())
166     with open(path, "w", encoding="utf-8-sig", newline="") as f:
167         w = csv.DictWriter(f, fieldnames=keys,
                ↪ extrasaction="ignore"); w.writeheader();
                ↪ w.writerows(rows)
168
169 def main():
170     if not os.path.isfile(OSM_PATH): raise
            ↪ FileNotFoundError("map.osm not found")
171     os.makedirs(OUT_DIR, exist_ok=True)
172     nodes, node_tags, ways_nds, way_tags, relations,
            ↪ relation_members, relation_tags = parse_osm()
```

```
    write_boundaries(relation_tags, way_tags); write_roads(ways_nds,
        ↪ way_tags, nodes)
    write_gates(nodes, node_tags, ways_nds, way_tags);
        ↪ write_waterholes(nodes, node_tags, ways_nds, way_tags)
    write_camps_facilities(nodes, node_tags, ways_nds, way_tags);
        ↪ write_terrain(way_tags, relation_tags, ways_nds, nodes);
        ↪ write_fence_restricted(way_tags, node_tags, ways_nds,
        ↪ nodes)

if __name__ == "__main__": main()
```

A.2 Task 1: Grid and Risk

Listing A.2: Grid and Risk

```
import os
import csv
import xml.etree.ElementTree as ET
from collections import defaultdict
import math
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.path import Path

CODE_DIR = os.path.dirname(os.path.abspath(__file__))
OSM_PATH = os.path.join(CODE_DIR, "map.osm")
OUT_DIR = os.path.join(CODE_DIR, "output")
BOUNDARY_RELATIONS = {"2982497", "5694335", "14066259"}
BOUNDARY_WAYS = ["863042084"]
GRID_SIZE_DEG = 0.04
RISK_THRESHOLD_TAU = 0.6

def chain_ways(way_refs, ways_nds):
    refs = list(way_refs); segs = [list(ways_nds.get(wid, [])) for
        ↪ wid in refs]; segs = [s for s in segs if s]
    if not segs: return []
    ring = list(segs[0]); used = {0}
    while len(used) < len(segs):
        last, first = ring[-1], ring[0]; found = False
        for i, s in enumerate(segs):
            if i in used: continue
            if s[0] == last: ring.extend(s[1:]); used.add(i); found
                ↪ = True; break
            if s[-1] == last: ring.extend(s[-2::-1]); used.add(i);
                ↪ found = True; break
            if s[-1] == first: ring[:0] = s[:-1]; used.add(i); found
                ↪ = True; break
            if s[0] == first: ring[:0] = s[1:]; used.add(i); found =
                ↪ True; break
        if not found: break
    if ring and ring[0] != ring[-1]: ring.append(ring[0])
    return ring

def parse_osm_boundaries():
    nodes, ways_nds, rel_members = {}, {}, {}
    for event, elem in ET.iterparse(OSM_PATH, events=("end",)):
```

```
        if elem.tag == "node":
            nid, lat, lon = elem.get("id"), elem.get("lat"),
                ↪ elem.get("lon")
            if nid and lat and lon: nodes[nid] = (float(lat),
                ↪ float(lon))
            elem.clear()
        elif elem.tag == "way":
            wid = elem.get("id")
            if wid: ways_nds[wid] = [nd.get("ref") for nd in
                ↪ elem.findall("nd") if nd.get("ref")]
            elem.clear()
        elif elem.tag == "relation":
            rid = elem.get("id")
            if rid in BOUNDARY_RELATIONS: rel_members[rid] =
                ↪ [(m.get("type"), m.get("ref"), m.get("role", ""))
                ↪ for m in elem.findall("member")]
            elem.clear()
    return nodes, ways_nds, rel_members

def build_polygons(nodes, ways_nds, rel_members):
    def coords(ring_refs): return [(nodes[ref][1], nodes[ref][0])
        ↪ for ref in ring_refs if ref in nodes]
    def one_ring(rid):
        if rid not in rel_members: return None
        outer_refs = [m[1] for m in rel_members[rid] if m[0] ==
            ↪ "way" and m[2] == "outer"]
        ring_refs = chain_ways(outer_refs, ways_nds)
        if not ring_refs: ring_refs = [n for ref in outer_refs for n
            ↪ in ways_nds.get(ref, [])]
        if ring_refs and ring_refs[0] == ring_refs[-1]: ring_refs =
            ↪ ring_refs[:-1]
        pts = coords(ring_refs)
        if len(pts) >= 3: return pts + [pts[0]] if pts[0] != pts[-1]
            ↪ else pts
        return None
    polygons = []
    for rid in ["2982497", "14066259"]:
        pts = one_ring(rid); if pts: polygons.append(("Etosha
            ↪ National Park", pts))
    for wid in BOUNDARY_WAYS:
        nds = list(ways_nds.get(wid, [])); nds = nds[:-1] if nds and
            ↪ nds[0] == nds[-1] else nds
        pts = coords(nds); polygons.append(("Etosha National Park",
            ↪ pts + [pts[0]])) if len(pts) >= 3 else None
    pan_pts = one_ring("5694335"); pan_pts and
        ↪ polygons.append(("Etosha Pan", pan_pts))
    return polygons

def point_in_polygon(lon, lat, ring): return
    ↪ Path(ring).contains_point((lon, lat))

def dist_km(lon1, lat1, lon2, lat2):
    lat_mid = (lat1 + lat2) * 0.5 * math.pi / 180
    dy, dx = 111 * (lat2 - lat1), 111 * math.cos(lat_mid) * (lon2 -
        ↪ lon1)
    return math.sqrt(dx*dx + dy*dy)

def point_to_segment_dist_km(px, py, x1, y1, x2, y2):
    dx, dy = x2 - x1, y2 - y1
```

```
    if dx == 0 and dy == 0: return dist_km(px, py, x1, y1)
    t = max(0, min(1, ((px - x1) * dx + (py - y1) * dy) / (dx*dx +
        ↪ dy*dy)))
    return dist_km(px, py, x1 + t * dx, y1 + t * dy)

def min_dist_to_polyline(lon, lat, segments_by_way, max_ways=500):
    d = float("inf")
    for wid, pts in list(segments_by_way.items())[:max_ways]:
        if len(pts) < 2: continue
        for i in range(len(pts) - 1): d = min(d,
            ↪ point_to_segment_dist_km(lon, lat, pts[i][0],
            ↪ pts[i][1], pts[i+1][0], pts[i+1][1]))
    return d if d != float("inf") else 50.0

def load_points(path, lon_key="lon", lat_key="lat"):
    out = []
    if not os.path.isfile(path): return out
    with open(path, "r", encoding="utf-8-sig") as f:
        for row in csv.DictReader(f):
            try: out.append((float(row.get(lon_key, row.get("lon",
                ↪ ""))), float(row.get(lat_key, row.get("lat",
                ↪ ""))))
            except (ValueError, TypeError): continue
    return out

def load_roads_geometry(path):
    by_way = defaultdict(list)
    if not os.path.isfile(path): return by_way
    with open(path, "r", encoding="utf-8-sig") as f:
        for row in csv.DictReader(f):
            try: lon, lat = float(row.get("lon", "")),
                ↪ float(row.get("lat", "")); wid, seq =
                ↪ row.get("way_id", ""), int(row.get("sequence", 0))
            except (ValueError, TypeError): continue
            if wid: by_way[wid].append((seq, lon, lat))
    for wid in by_way: by_way[wid].sort(key=lambda x: x[0]);
        ↪ by_way[wid] = [(x[1], x[2]) for x in by_way[wid]]
    return dict(by_way)

def main():
    if not os.path.isdir(OUT_DIR) or not os.path.isfile(OSM_PATH):
        ↪ raise FileNotFoundError("Run data processing first and
        ↪ ensure map.osm exists")
    nodes, ways_nds, rel_members = parse_osm_boundaries(); polygons
        ↪ = build_polygons(nodes, ways_nds, rel_members)
    park_rings = [ring for name, ring in polygons if name == "Etosha
        ↪ National Park"]; pan_ring = next((ring for name, ring in
        ↪ polygons if name == "Etosha Pan"), None)
    all_lons = [x for _, ring in polygons for x, y in ring];
        ↪ all_lats = [y for _, ring in polygons for x, y in ring]
    lon_min, lon_max = min(all_lons), max(all_lons); lat_min,
        ↪ lat_max = min(all_lats), max(all_lats)
    half = GRID_SIZE_DEG / 2; cells = []; gid = 0
    lon_c = lon_min + half
    while lon_c <= lon_max:
        lat_c = lat_min + half
        while lat_c <= lat_max:
            in_park = any(point_in_polygon(lon_c, lat_c, ring) for
                ↪ ring in park_rings)
```

```
            if in_park:
                in_pan = point_in_polygon(lon_c, lat_c, pan_ring) if pan_ring else False
                cells.append({"grid_id": gid, "lon": lon_c, "lat": lat_c, "in_pan": in_pan}); gid += 1
            lat_c += GRID_SIZE_DEG
        lon_c += GRID_SIZE_DEG
    gates = load_points(os.path.join(OUT_DIR, "gates.csv")); camps = load_points(os.path.join(OUT_DIR, "camps_facilities.csv"))
    waterholes = load_points(os.path.join(OUT_DIR, "waterholes.csv")); roads = load_roads_geometry(os.path.join(OUT_DIR, "roads_geometry.csv"))
    for c in cells:
        lon, lat = c["lon"], c["lat"]
        c["d_road_km"] = min_dist_to_polyline(lon, lat, roads); c["d_gate_km"] = min((dist_km(lon, lat, g[0], g[1]) for g in gates), default=50.0)
        c["d_camp_km"] = min((dist_km(lon, lat, p[0], p[1]) for p in camps), default=50.0); c["d_water_km"] = min((dist_km(lon, lat, w[0], w[1]) for w in waterholes), default=50.0)
        c["r_human"] = 1.0/(1.0+c["d_road_km"]/8)*0.6 + 1.0/(1.0+c["d_gate_km"]/15)*0.4
        c["r_nat"] = 0.4 + 0.6 * (1.0 if c["in_pan"] else 0.3); c["r_species"] = (0.7 if c["in_pan"] else 0.4) + 0.3/(1.0+c["d_water_km"]/12)
        c["r_coverage"] = 1.0/(1.0+min(c["d_road_km"], c["d_camp_km"])/10)
        for k in ["r_human", "r_nat", "r_species", "r_coverage"]: c[k] = min(1.0, max(0.0, c[k]))
        scores = [c["r_human"], c["r_nat"], c["r_species"], c["r_coverage"]]; c["main_risk"] = ["H", "N", "S", "C"][scores.index(max(scores))]
        c["priority"] = (c["r_human"]+c["r_nat"]+c["r_species"])*(1.0+0.5*c["r_coverage"])
        c["risk_combined"] = (c["r_human"]+c["r_nat"]+c["r_species"])/3.0; c["protection_level"] = round(1.0-c["risk_combined"], 4)
        c["protected"] = 1 if c["risk_combined"] <= RISK_THRESHOLD_TAU else 0
    keys = ["grid_id", "lon", "lat", "in_pan", "d_road_km", "d_gate_km", "d_camp_km", "d_water_km", "r_human", "r_nat", "r_species", "r_coverage", "main_risk", "priority", "risk_combined", "protection_level", "protected"]
    csv_path = os.path.join(OUT_DIR, "grid_risk.csv")
    with open(csv_path, "w", encoding="utf-8-sig", newline="") as f: w = csv.DictWriter(f, fieldnames=keys, extrasaction="ignore"); w.writeheader(); w.writerows(cells)
    from matplotlib.lines import Line2D
    risk_colors = {"H": "#c62828", "N": "#f9a825", "S": "#2e7d32", "C": "#1565c0"}
    xlim = (lon_min - 0.08, lon_max + 0.08); ylim = (lat_min - 0.06, lat_max + 0.06)
    def add_boundary(ax):
        ax.set_facecolor("#fafafa")
        for name, ring in polygons:
```

```
            ↪ ax.add_patch(mpatches.Polygon([(x,y) for x,y in ring],
            ↪ facecolor="none", edgecolor="#2d5a27", linewidth=0.8))
151         ax.set_xlim(xlim); ax.set_ylim(ylim);
            ↪ ax.set_aspect("equal"); ax.grid(True, linestyle="--",
            ↪ alpha=0.4)
152     fig1, ax1 = plt.subplots(figsize=(8, 6), facecolor="#fafafa");
        ↪ add_boundary(ax1)
153     for c in cells: ax1.scatter(c["lon"], c["lat"],
        ↪ c=risk_colors.get(c["main_risk"], "gray"), s=14,
        ↪ alpha=0.75, edgecolors="none")
154     ax1.set_title("Q1: Main risk type per grid"); leg1 =
        ↪ [Line2D([0],[0], marker="o", color="w",
        ↪ markerfacecolor=risk_colors[k], label=k, markersize=8) for
        ↪ k in ["H","N","S","C"]]
155     ax1.legend(handles=leg1, loc="upper left", fontsize=8);
        ↪ plt.tight_layout(); plt.savefig(os.path.join(OUT_DIR,
        ↪ "q1_main_risk.pdf"), format="pdf", bbox_inches="tight",
        ↪ facecolor="#fafafa"); plt.close()
156     fig2, ax2 = plt.subplots(figsize=(8, 6), facecolor="#fafafa");
        ↪ add_boundary(ax2)
157     for c in cells: ax2.scatter(c["lon"], c["lat"], c="#2e7d32" if
        ↪ c["protected"] else "#e0e0e0", s=14, alpha=0.75,
        ↪ edgecolors="none")
158     ax2.set_title("Q1: Protected (green) vs not (gray)");
        ↪ ax2.legend(handles=[Line2D([0],[0], marker="o", color="w",
        ↪ markerfacecolor="#2e7d32", label="Protected",
        ↪ markersize=8), Line2D([0],[0], marker="o", color="w",
        ↪ markerfacecolor="#e0e0e0", label="Not protected",
        ↪ markersize=8)], loc="upper left"); plt.tight_layout();
        ↪ plt.savefig(os.path.join(OUT_DIR, "q1_protected.pdf"),
        ↪ format="pdf", bbox_inches="tight", facecolor="#fafafa");
        ↪ plt.close()
159
160 if __name__ == "__main__": main()
```

A.3 Task 2: Resource Allocation

Listing A.3: Greedy allocation by dominant risk and priority

```
1  import os
2  import csv
3  import xml.etree.ElementTree as ET
4  from collections import defaultdict
5  import matplotlib.pyplot as plt
6  import matplotlib.patches as mpatches
7  from matplotlib.lines import Line2D
8
9  CODE_DIR = os.path.dirname(os.path.abspath(__file__)); OUT_DIR =
       ↪ os.path.join(CODE_DIR, "output"); OSM_PATH =
       ↪ os.path.join(CODE_DIR, "map.osm")
10 BOUNDARY_RELATIONS = {"2982497", "5694335", "14066259"};
       ↪ BOUNDARY_WAYS = ["863042084"]
11 WEIGHT = {"H": {"patrol": 0.9, "fence": 1.0, "monitoring": 0.8},
       ↪ "N": {"patrol": 0.5, "fence": 0.2, "monitoring": 0.9}, "S":
       ↪ {"patrol": 1.0, "fence": 0.5, "monitoring": 0.9}, "C":
       ↪ {"patrol": 0.3, "fence": 0.2, "monitoring": 1.0}}
```

```
BUDGET = {"patrol": 120, "fence": 80, "monitoring": 60}

def load_grid_risk(path):
    cells = []
    with open(path, "r", encoding="utf-8-sig") as f:
        for row in csv.DictReader(f):
            try: row["lon"]=float(row["lon"]); row["lat"]=float(row["lat"]); row["priority"]=float(row["priority"]); row["grid_id"]=int(row["grid_id"]); cells.append(row)
            except (ValueError, TypeError, KeyError): continue
    return cells

def _chain_ways(way_refs, ways_nds):
    refs = list(way_refs); segs = [list(ways_nds.get(wid, [])) for wid in refs]; segs = [s for s in segs if s]
    if not segs: return []
    ring = list(segs[0]); used = {0}
    while len(used) < len(segs):
        last, first = ring[-1], ring[0]; found = False
        for i, s in enumerate(segs):
            if i in used: continue
            if s[0]==last: ring.extend(s[1:]); used.add(i); found=True; break
            if s[-1]==last: ring.extend(s[-2::-1]); used.add(i); found=True; break
            if s[-1]==first: ring[:0]=s[:-1]; used.add(i); found=True; break
            if s[0]==first: ring[:0]=s[1:]; used.add(i); found=True; break
        if not found: break
    if ring and ring[0]!=ring[-1]: ring.append(ring[0])
    return ring

def _parse_osm_boundaries():
    nodes, ways_nds, rel_members = {}, {}, {}
    for event, elem in ET.iterparse(OSM_PATH, events=("end",)):
        if elem.tag == "node": nid, lat, lon = elem.get("id"), elem.get("lat"), elem.get("lon"); (nodes.__setitem__(nid, (float(lat), float(lon))) if nid and lat and lon else None); elem.clear()
        elif elem.tag == "way": wid = elem.get("id"); (ways_nds.__setitem__(wid, [nd.get("ref") for nd in elem.findall("nd") if nd.get("ref")]) if wid else None); elem.clear()
        elif elem.tag == "relation": rid = elem.get("id"); (rel_members.__setitem__(rid, [(m.get("type"), m.get("ref"), m.get("role","")) for m in elem.findall("member")]) if rid in BOUNDARY_RELATIONS else None); elem.clear()
    return nodes, ways_nds, rel_members

def _build_polygons(nodes, ways_nds, rel_members):
    def coords(ring_refs): return [(nodes[ref][1], nodes[ref][0]) for ref in ring_refs if ref in nodes]
    def one_ring(rid):
        if rid not in rel_members: return None
        outer_refs = [m[1] for m in rel_members[rid] if m[0]=="way"
```

```
            ↪ and m[2]=="outer"]; ring_refs =
            ↪ _chain_ways(outer_refs, ways_nds)
        if not ring_refs: ring_refs = [n for ref in outer_refs for n
            ↪ in ways_nds.get(ref, [])]
        if ring_refs and ring_refs[0]==ring_refs[-1]: ring_refs =
            ↪ ring_refs[:-1]
        pts = coords(ring_refs); return (pts + [pts[0]] if
            ↪ len(pts)>=3 and pts[0]!=pts[-1] else pts) if
            ↪ len(pts)>=3 else None
    polygons = []
    for rid in ["2982497", "14066259"]: pts = one_ring(rid); pts and
        ↪ polygons.append(("Etosha National Park", pts))
    for wid in BOUNDARY_WAYS: nds = list(ways_nds.get(wid, [])); nds
        ↪ = nds[:-1] if nds and nds[0]==nds[-1] else nds; pts =
        ↪ coords(nds); (polygons.append(("Etosha National Park",
        ↪ pts+[pts[0]])) if len(pts)>=3 else None)
    pan_pts = one_ring("5694335"); pan_pts and
        ↪ polygons.append(("Etosha Pan", pan_pts))
    return polygons

def load_boundary_polygons():
    if not os.path.isfile(OSM_PATH): return []
    try: nodes, ways_nds, rel_members = _parse_osm_boundaries();
        ↪ return _build_polygons(nodes, ways_nds, rel_members)
    except Exception: return []

def main():
    risk_path = os.path.join(OUT_DIR, "grid_risk.csv")
    if not os.path.isfile(risk_path) or not os.path.isdir(OUT_DIR):
        ↪ raise FileNotFoundError("Run q1_grid_risk.py and data
        ↪ processing first")
    cells = load_grid_risk(risk_path); sorted_cells = sorted(cells,
        ↪ key=lambda x: -x["priority"]); remaining = dict(BUDGET);
        ↪ allocation = []
    for c in sorted_cells:
        gid, main = c["grid_id"], c.get("main_risk", "C"); w =
            ↪ WEIGHT.get(main, WEIGHT["C"]); best = sorted(w.keys(),
            ↪ key=lambda r: -w[r])
        for res in best:
            if remaining.get(res, 0) <= 0: continue
            remaining[res] = remaining.get(res, 0) - 1;
                ↪ allocation.append({"grid_id": gid, "lon":
                ↪ c["lon"], "lat": c["lat"], "main_risk": main,
                ↪ "resource": res, "amount": 1}); break
    with open(os.path.join(OUT_DIR, "grid_allocation.csv"), "w",
        ↪ encoding="utf-8-sig", newline="") as f: w =
        ↪ csv.DictWriter(f,
        ↪ fieldnames=["grid_id","lon","lat","main_risk","resource","amount"]);
        ↪ w.writeheader(); w.writerows(allocation)
    polygons = load_boundary_polygons(); all_lons = [c["lon"] for c
        ↪ in cells]; all_lats = [c["lat"] for c in cells]
    lon_min, lon_max = min(all_lons), max(all_lons); lat_min,
        ↪ lat_max = min(all_lats), max(all_lats)
    margin_lon = max(0.08, (lon_max-lon_min)*0.05); margin_lat =
        ↪ max(0.06, (lat_max-lat_min)*0.05)
    risk_colors = {"H": "#c62828", "N": "#f9a825", "S": "#2e7d32",
        ↪ "C": "#1565c0"}; res_mark = {"patrol": "o", "fence": "s",
        ↪ "monitoring": "^"}
    fig, ax = plt.subplots(figsize=(10, 8), facecolor="#fafafa");
```

```
        ↪ ax.set_facecolor("#fafafa")
    for name, ring in polygons:
        xy = [(x,y) for x,y in ring]
        if name == "Etosha Pan": ax.add_patch(mpatches.Polygon(xy,
            ↪ facecolor="#e8e8e8", edgecolor="#9e9e9e",
            ↪ linewidth=0.6))
        else: ax.add_patch(mpatches.Polygon(xy, facecolor="#c8c0a0",
            ↪ edgecolor="#2d5a27", linewidth=0.8))
    for a in allocation: ax.scatter(a["lon"], a["lat"],
        ↪ c=risk_colors.get(a["main_risk"], "gray"), s=36,
        ↪ marker=res_mark.get(a["resource"], "o"), alpha=0.85,
        ↪ edgecolors="white", linewidths=0.3)
    ax.set_xlim(lon_min - margin_lon, lon_max + margin_lon);
        ↪ ax.set_ylim(lat_min - margin_lat, lat_max + margin_lat);
        ↪ ax.set_aspect("equal")
    ax.set_title("Q2: Resource allocation (color=main risk,
        ↪ marker=resource)"); ax.set_xlabel("Longitude");
        ↪ ax.set_ylabel("Latitude"); ax.grid(True, linestyle="--",
        ↪ alpha=0.4)
    leg_risk = [Line2D([0],[0], marker="o", color="w",
        ↪ markerfacecolor=risk_colors[k], label="Risk "+k,
        ↪ markersize=8) for k in ["H","N","S","C"]]
    leg_res = [Line2D([0],[0], marker=res_mark[r], color="w",
        ↪ markerfacecolor="gray", label=r, markersize=8) for r in
        ↪ ["patrol","fence","monitoring"]]
    ax.legend(handles=leg_risk+leg_res, loc="upper left",
        ↪ fontsize=8); plt.tight_layout();
        ↪ plt.savefig(os.path.join(OUT_DIR, "q2_allocation.pdf"),
        ↪ format="pdf", bbox_inches="tight", facecolor="#fafafa");
        ↪ plt.close()

if __name__ == "__main__": main()
```

A.4 Task 3: Protection Dynamics

Listing A.4: ODE: protection-threat-effort

```
import os
import csv
try: import matplotlib.pyplot as plt; HAS_MATPLOTLIB = True
except ImportError: HAS_MATPLOTLIB = False

CODE_DIR = os.path.dirname(os.path.abspath(__file__)); OUT_DIR =
    ↪ os.path.join(CODE_DIR, "output")
STAFF_REF = 295
R_P, ALPHA, BETA, GAMMA, DELTA = 0.08, 0.15, 0.12, 0.10, 0.25

def load_initial_protection():
    path = os.path.join(OUT_DIR, "grid_risk.csv")
    if not os.path.isfile(path): raise FileNotFoundError("Run
        ↪ q1_grid_risk.py first")
    protected, risk_combined = [], []
    with open(path, "r", encoding="utf-8-sig") as f:
        for row in csv.DictReader(f):
            try: protected.append(int(row.get("protected", 0)));
                ↪ risk_combined.append(float(row.get("risk_combined",
```

```
                ↪ 0.5)))
17              except (ValueError, TypeError): continue
18      if not protected: return 0.5, 0.5
19      P0 = sum(protected)/len(protected); r_avg =
            ↪ sum(risk_combined)/len(risk_combined)
20      T0 = min(2.0, max(0.05, r_avg*(1.0-P0+0.1)))
21      return float(P0), float(T0)
22
23  def ode(t, y, E, r_P, alpha, beta, gamma, delta):
24      P, T = max(0.0, min(1.0, y[0])), max(0.0, y[1])
25      dP = r_P*(1.0-P) - alpha*T*P + beta*E; dT = gamma*T*(1.0-P) -
            ↪ delta*E*T
26      return [dP, dT]
27
28  def run_trajectory(E, t_span=(0, 50), y0=None, r_P=R_P, alpha=ALPHA,
        ↪ beta=BETA, gamma=GAMMA, delta=DELTA, n_steps=400):
29      if y0 is None: y0 = [0.55, 0.4]
30      t0, t1 = t_span[0], t_span[1]; dt = (t1-t0)/n_steps
31      t_list = [t0 + i*dt for i in range(n_steps+1)]; P_list, T_list =
            ↪ [max(0.0, min(1.0, y0[0]))], [max(0.0, y0[1])]
32      for i in range(n_steps):
33          P, T = P_list[-1], T_list[-1]; dP, dT = ode(t_list[i], [P,
                ↪ T], E, r_P, alpha, beta, gamma, delta)
34          P_list.append(max(0.0, min(1.0, P + dP*dt)));
                ↪ T_list.append(max(0.0, T + dT*dt))
35      return t_list, P_list, T_list
36
37  def steady_state_E_for_target_P(P_bar, r_P=R_P, alpha=ALPHA,
        ↪ beta=BETA, gamma=GAMMA, delta=DELTA):
38      if P_bar >= 1.0: return 0.0
39      return (gamma/delta)*(1.0 - P_bar)
40
41  def main():
42      os.makedirs(OUT_DIR, exist_ok=True); P0, T0 =
            ↪ load_initial_protection()
43      t_span = (0, 40); E_values = [0.3, 0.7, 1.0, 1.5]
44      if HAS_MATPLOTLIB:
45          fig, axes = plt.subplots(2, 1, figsize=(7, 5), sharex=True,
                ↪ facecolor="#fafafa"); ax_p, ax_t = axes[0], axes[1]
46          for E in E_values: t, P, T = run_trajectory(E,
                ↪ t_span=t_span, y0=[P0, T0]); ax_p.plot(t, P, label="E
                ↪ = %s (%s staff)" % (E, int(STAFF_REF*E)));
                ↪ ax_t.plot(t, T, label="E = %s" % E)
47          ax_p.set_ylabel("Protection level P(t)"); ax_p.set_ylim(0,
                ↪ 1.05); ax_p.legend(loc="lower right", fontsize=8);
                ↪ ax_p.grid(True, linestyle="--", alpha=0.5);
                ↪ ax_p.set_title("Q3: Protection and threat over time")
48          ax_t.set_ylabel("Threat pressure T(t)");
                ↪ ax_t.set_xlabel("Time (e.g. months)");
                ↪ ax_t.legend(loc="upper right", fontsize=8);
                ↪ ax_t.grid(True, linestyle="--", alpha=0.5)
49          plt.tight_layout(); plt.savefig(os.path.join(OUT_DIR,
                ↪ "q3_protection_over_time.pdf"), format="pdf",
                ↪ bbox_inches="tight", facecolor="#fafafa"); plt.close()
50      t1, P1, T1 = run_trajectory(1.0, t_span=t_span, y0=[P0, T0])
51      csv_path = os.path.join(OUT_DIR, "q3_protection_time_series.csv")
52      with open(csv_path, "w", encoding="utf-8-sig", newline="") as f:
            ↪ w = csv.DictWriter(f, fieldnames=["t","P","T","E"]);
            ↪ w.writeheader(); [w.writerow({"t": round(t1[i],2), "P":
```

```
        ↪ round(P1[i],4), "T": round(T1[i],4), "E": 1.0}) for i in
        ↪ range(0, len(t1), 2)]
    P_bars = [0.4 + i*0.025 for i in range(19)]; E_stars =
        ↪ [steady_state_E_for_target_P(p) for p in P_bars]; staffs =
        ↪ [int(STAFF_REF*E) for E in E_stars]
    if HAS_MATPLOTLIB:
        fig2, ax2 = plt.subplots(figsize=(6, 4),
            ↪ facecolor="#fafafa"); ax2.plot(P_bars, E_stars, "b-",
            ↪ linewidth=2, label="Required effort E*");
            ↪ ax2.set_xlabel("Target protection level P_bar");
            ↪ ax2.set_ylabel("Required effort E* (1 = current ~295
            ↪ staff)"); ax2.grid(True, linestyle="--", alpha=0.5);
            ↪ ax2.legend(loc="upper right"); ax2_r = ax2.twinx();
            ↪ ax2_r.plot(P_bars, staffs, "g--", linewidth=1.5,
            ↪ alpha=0.8, label="Estimated staff");
            ↪ ax2_r.set_ylabel("Estimated personnel");
            ↪ ax2_r.legend(loc="center right"); ax2.set_title("Q3:
            ↪ Staffing need to maintain target protection (steady
            ↪ state)"); ax2.set_xlim(0.35, 0.9); plt.tight_layout();
            ↪ plt.savefig(os.path.join(OUT_DIR,
            ↪ "q3_staffing_vs_protection.pdf"), format="pdf",
            ↪ bbox_inches="tight", facecolor="#fafafa"); plt.close()

if __name__ == "__main__": main()
```

A.5 Task 4: Sensitivity

Listing A.5: Sensitivity of E* to ODE params

```
import os
import csv
from q3_protection_dynamics import load_initial_protection,
    ↪ run_trajectory, steady_state_E_for_target_P, OUT_DIR,
    ↪ STAFF_REF, R_P, ALPHA, BETA, GAMMA, DELTA
try: import matplotlib.pyplot as plt; HAS_MATPLOTLIB = True
except ImportError: HAS_MATPLOTLIB = False

P_BAR_TARGET = 0.6

def run_sensitivity():
    base = {"r_P": R_P, "alpha": ALPHA, "gamma": GAMMA, "delta":
        ↪ DELTA}; rows = []
    for param_name, base_val in base.items():
        for scale in [0.8, 1.0, 1.2]:
            val = base_val * scale; kwargs = {"r_P": R_P, "alpha":
                ↪ ALPHA, "beta": BETA, "gamma": GAMMA, "delta":
                ↪ DELTA}; kwargs[param_name] = val
            E_star = steady_state_E_for_target_P(P_BAR_TARGET,
                ↪ **kwargs); staff = int(STAFF_REF * E_star)
            rows.append({"param": param_name, "scale": scale,
                ↪ "value": round(val, 4), "E_star": round(E_star,
                ↪ 4), "staff": staff})
    return rows

def run_scenarios(P0, T0, t_span=(0, 40)):
    return [(E, *run_trajectory(E, t_span=t_span, y0=[P0, T0])[:2])
```

```
        ↪ for E in [0.5, 0.7, 1.0]]

def main():
    os.makedirs(OUT_DIR, exist_ok=True); P0, T0 =
        ↪ load_initial_protection()
    sens_rows = run_sensitivity(); csv_path = os.path.join(OUT_DIR,
        ↪ "q4_sensitivity_params.csv")
    with open(csv_path, "w", encoding="utf-8-sig", newline="") as f:
        ↪ w = csv.DictWriter(f,
        ↪ fieldnames=["param","scale","value","E_star","staff"]);
        ↪ w.writeheader(); w.writerows(sens_rows)
    if HAS_MATPLOTLIB:
        params, x_labels = ["r_P", "alpha", "gamma", "delta"],
            ↪ ["r_P", "alpha", "gamma", "delta"]
        fig1, axes = plt.subplots(2, 2, figsize=(7, 5),
            ↪ facecolor="#fafafa")
        for idx, (p, lbl) in enumerate(zip(params, x_labels)):
            ax = axes[idx//2, idx%2]; sub = [r for r in sens_rows if
                ↪ r["param"]==p]; ax.plot([r["scale"] for r in sub],
                ↪ [r["E_star"] for r in sub], "o-", linewidth=2,
                ↪ markersize=8); ax.set_xlabel("%s (scale)" % lbl);
                ↪ ax.set_ylabel("E*"); ax.set_title("Sensitivity to
                ↪ %s" % lbl); ax.grid(True, linestyle="--",
                ↪ alpha=0.5)
        plt.suptitle("Q4: Sensitivity of required effort E* to
            ↪ parameters (P_bar=0.6)"); plt.tight_layout();
            ↪ plt.savefig(os.path.join(OUT_DIR,
            ↪ "q4_sensitivity.pdf"), format="pdf",
            ↪ bbox_inches="tight", facecolor="#fafafa"); plt.close()
    scenarios = run_scenarios(P0, T0); labels = {0.5: "E=0.5 (~148
        ↪ staff)", 0.7: "E=0.7 (~207 staff)", 1.0: "E=1.0 (baseline
        ↪ ~295 staff)"}
    if HAS_MATPLOTLIB:
        fig2, ax2 = plt.subplots(figsize=(6, 4), facecolor="#fafafa")
        for E, t, P in scenarios: ax2.plot(t, P, label=labels.get(E,
            ↪ "E=%s" % E))
        ax2.set_xlabel("Time (e.g. months)");
            ↪ ax2.set_ylabel("Protection level P(t)");
            ↪ ax2.set_title("Q4: Scenario - protection over time
            ↪ under reduced personnel"); ax2.legend(loc="lower
            ↪ right", fontsize=8); ax2.set_ylim(0, 1.05);
            ↪ ax2.grid(True, linestyle="--", alpha=0.5);
            ↪ plt.tight_layout(); plt.savefig(os.path.join(OUT_DIR,
            ↪ "q4_scenarios.pdf"), format="pdf",
            ↪ bbox_inches="tight", facecolor="#fafafa"); plt.close()

if __name__ == "__main__": main()
```

B Use of AI

The team used AI tools to clarify concepts and formulas for the methods used in this paper. Below we record representative questions and summarized AI responses. All model design, equations, parameter choices, coding, and writing were done by the team; AI was used only for background understanding.

Question: Lotka-Volterra model and ODE for protection–threat dynamics

What is the Lotka-Volterra model? How can we adapt it to describe how protection level and threat pressure evolve over time in a wildlife reserve, with staffing (effort) as a control?

AI Response

The Lotka-Volterra model describes predator–prey dynamics: prey grow in absence of predators; predators consume prey and grow when prey are abundant. In conservation, one can treat *protection* as "prey" (eroded by threat) and *threat* as "predator" (growing when protection is low). Adding an external *effort* E (e.g., staff, technology) that boosts protection and suppresses threat gives a protection–threat–effort ODE:

$$\frac{dP}{dt} = r_P(1-P) - \alpha TP + \beta E, \qquad \frac{dT}{dt} = \gamma T(1-P) - \delta ET.$$

Here $P(t)$ is protection level, $T(t)$ is threat pressure, r_P is natural recovery of protection, α is erosion by threat, β is gain from effort, γ is threat growth when protection is low, and δ is suppression of threat by effort. At equilibrium ($dP/dt = dT/dt = 0$), the effort required to maintain a target $\bar{P}$ is $E^* = (\gamma/\delta)(1-\bar{P})$, which we use for staffing estimates.

Question: Spatial conservation prioritization and risk-based protection definition

How can we prioritize conservation areas using a grid and multiple risk dimensions, and define "protection" in a measurable way?

AI Response

Spatial conservation prioritization often uses a grid over the area and assigns each cell scores for different risk or value dimensions (e.g., human access, natural hazards, species vulnerability, patrol coverage). Each dimension can be normalized to $[0, 1]$. The *dominant risk type* for a cell is the dimension with the highest score (e.g., H/N/S/C). A *combined risk* can be defined as a weighted average of selected dimensions, and *protection level* as $1 -$ risk. Then "protected" is defined by a threshold τ: a cell is protected if risk $\leq \tau$. Priority for resource allocation can combine the risk scores and coverage gap so that high-risk and hard-to-reach cells get higher priority. This yields a quantitative, map-based definition of protection and priorities.

Question: Allocating limited resources (patrol, fence, monitoring) across a protected area

How should we allocate limited protection resources (e.g., ranger patrols, fencing, monitoring technology) across many grid cells when different resource types are more effective for different types of risk?

AI Response

A common approach is to (1) assign each cell a *dominant risk type* and a *priority* (e.g., from risk and coverage gap); (2) define a *risk–resource weight matrix* w_{rk}: effectiveness of resource k when dominant risk is r; (3) maximize total weighted protection subject to budget constraints (e.g., $\sum_i x_{ik} \leq B_k$ for each resource k). When the problem is large or discrete, a *greedy* rule is often used: sort cells by priority (descending), and for each cell allocate the resource with the highest $w_{r(i),k}$ among those with remaining budget. This matches resource type to risk type and emphasizes high-priority cells, yielding a strategic (rather than uniform) deployment that can be visualized on a map.

Question: Sensitivity analysis and scenario analysis for ODE parameters and effort

How do we assess how sensitive the required effort (e.g., staffing) is to the ODE parameters, and how do we compare protection over time under different effort scenarios?

AI Response

Sensitivity analysis: Fix a target protection level $\bar{P}$ and the steady-state formula $E^* = (\gamma/\delta)(1 - \bar{P})$. Vary each parameter (e.g., γ, δ) by scale factors (e.g., 0.8, 1.0, 1.2) and recompute E^*. Plot E^* versus the scale factor for each parameter to see which parameters most affect required effort; parameters not appearing in E^* leave steady-state staffing unchanged. **Scenario analysis:** Fix several effort levels E (e.g., baseline, -30%, -50% staff). For each E, solve the ODE from the same initial conditions and plot $P(t)$ over time. Compare how quickly protection declines and to what equilibrium under reduced effort, to quantify the impact of resource cuts and support minimum-staffing or technology-substitution arguments.